TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

GUIDE

(Guided User Itinerary & Destination Explorer)

Low-Level Design Report

QUAD-CORE Team

Ebrar Sude Doğan
Erdem Baran
Kayrahan Toprak Tosun
Tuna Kodal

Contents

1 Introduction **4**
1.1 Traceability matrix (HLD → LLD mapping) 5
1.2 Object design trade-offs 6
1.2.1 Usability vs. Functionality 7
1.2.2 Efficiency vs. Accuracy 7
1.2.3 Security vs. Usability 7
1.2.4 Reliability vs. Compatibility 7
1.2.5 Automation vs. User Control 8
1.3 Interface documentation guidelines 8
1.4 Engineering standards 10
1.5 Definitions, acronyms, and abbreviations 11

2 Packages **11**
2.1 Backend Packages 12
2.1.1 API Boundary 12
2.1.2 Core Services 13
2.1.3 Integration 13
2.1.4 Data Access 13
2.1.5 Shared Model 14
2.1.6 Presentation 14
2.2 Frontend Packages 14
2.2.1 Entry 15
2.2.2 Global Styles 15
2.2.3 Layout Modules 15
2.2.4 Page Modules 15
2.2.5 UI Modules 16
2.2.6 API Client 16
2.2.7 Utilities 16

3 Class Interfaces **16**

4 Behavioral Design **18**
4.1 Activity Models 18
4.1.1 Complete Route Planning Flow 19
4.1.2 POI Information Retrieval Flow 19
4.2 Sequence Models 20
4.2.1 Input Collection Sequence Diagram 20
4.2.2 POI Details and Voice Guidance Display 21
4.2.3 Route Generation and Replanning Sequence 22

5 Data Structures **23**

5.1 Data Transfer Objects . . . 23
5.2 Data Types and Size Considerations . . . 24
5.2.1 General conventions . . . 24
5.2.2 Core data structures and field types . . . 25
5.2.3 Recommended validation limits . . . 26
5.2.4 API Payload Size Considerations . . . 27
5.2.5 Media Size Considerations . . . 27
5.2.6 Textual Content Size Consideration . . . 28

6 Constraints 29
6.1 Technical Constraints . . . 29
6.2 Infrastructure and Deployment Constraints . . . 31
6.3 Functional Scope Constraints . . . 31
6.4 Academic and Development Constraints . . . 33

7 Error Conditions and Edge Cases 33
7.1 Input and Validation Errors . . . 33
7.2 Planning and Feasibility Edge Cases . . . 34
7.3 Integration and Routing Failures . . . 34
7.4 Data Availability and Content Gaps . . . 34
7.5 Robustness Strategy . . . 35

8 Testability Considerations 35
8.1 Unit-Testable Components . . . 35
8.2 Ranking Evaluation Challenges . . . 36
8.3 Mockable Dependencies . . . 36
8.4 Integration Testing Scope . . . 36
8.5 Test Coverage Expectations . . . 37

9 Glossary 38

A REST API Contract 38
A.1 Endpoint Overview . . . 39
A.2 HTTP Status Codes . . . 39
A.3 Sample Request and Response Payloads . . . 40
A.3.1 POST /api/v1/routes/generate . . . 40
A.3.2 POST /api/v1/routes/replan . . . 41
A.3.3 POST /api/v1/pois/search . . . 41
A.3.4 POST /api/v1/pois/content . . . 42

B Detailed Class Interfaces 43
B.1 Backend . . . 43
B.1.1 API Boundary . . . 43
B.1.2 Core Services . . . 45

B.1.3 Integration 47
B.1.4 Data Access 48
B.1.5 Data Transfer Objects and Core Data Structures (Shared Model Package) 49
B.1.6 Presentation 52
B.1.7 Additional DTOs, Enums, and Supporting Types 53

References **57**

1 Introduction

Travel planning has become increasingly fragmented with the rise of digital tools. When users plan a trip, they often rely on multiple disconnected applications such as map services, blog posts, review platforms, and navigation tools. This process is time-consuming and requires significant manual effort to organize destinations into a coherent and time-efficient itinerary. In addition, many existing systems do not consider user-specific constraints such as daily travel limits, category preferences, or visit duration balance.

The idea behind GUIDE (Guided User Itinerary & Destination Explorer) emerged from this problem. The main motivation of the project is to create a unified system that transforms user preferences into structured, optimized, multi-day travel itineraries. Instead of manually searching and arranging points of interest, users provide high-level constraints such as city, trip duration, maximum travel distance, and preferred categories. The system then generates an itinerary by filtering relevant POIs, estimating visit durations, and calculating feasible routes using OpenStreetMap and OSRM [1, 2].

Another important motivation for GUIDE is accessibility and contextual experience. In addition to route planning, the system provides descriptive information and multilingual text-to-speech audio guidance for each destination. This enables users to not only navigate efficiently, but also understand the cultural and historical context of the places they visit.

This report describes the low-level architecture and detailed design of GUIDE. It refines the High-Level Design (HLD) document by translating architectural components into implementation-oriented packages, classes, interfaces, and workflows [3]. The document covers object design trade-offs, interface documentation guidelines, engineering standards, package decomposition, and detailed class interface definitions. In addition, it includes structural and behavioral models that describe key system workflows such as route generation, replanning, and POI content retrieval. The goal of this report is to serve as an implementation-level blueprint that ensures consistency between system requirements, high-level architecture, and the final codebase.

Original contribution:

Although itinerary planning is a well-studied problem, GUIDE introduces a practically oriented integration of multiple components that are typically handled in isolation. The system combines

(i) a manually curated and semantically enriched POI dataset focused on Türkiye,

(ii) a Monte Carlo-based candidate generation and heuristic ranking strategy tailored for multi-day trip planning,

(iii) a stateless replanning mechanism that allows iterative user-driven modifications without maintaining server-side session state, and

(iv) offline-prepared multilingual content and audio guidance for each POI.

This combination enables GUIDE to function not only as a route generator but also as a context-aware travel assistant, integrating routing, content delivery, and interactive replanning within a unified system.

1.1 Traceability matrix (HLD → LLD mapping)

Table 1 presents the traceability matrix that establishes a structured relationship between system requirements, High-Level Design (HLD) layers, and their concrete realizations in the Low-Level Design (LLD). The matrix illustrates how each functional, interface, and performance requirement is addressed across the architectural layers and implemented through specific packages and classes.

The first column groups related requirements that are primarily handled together at the architectural level, while the second and third columns indicate the corresponding HLD layer and the LLD package are responsible for fulfilling these requirements. The final column lists the concrete LLD classes that realize the required behavior, enabling a direct mapping from abstract system requirements to implementation-level components.

This organization supports requirement coverage analysis and design verification by making explicit which architectural elements are responsible for each requirement. Detailed requirement definitions are provided in the Project Specifications Report, while the architectural responsibilities of subsystems are defined in the High-Level Design (HLD) Report [4].

Related Requirements	HLD Layer	LLD Package	LLD Classes/Enums
FR-H01, FR-L01, FR-H03–FR-L05, FR-H06, FR-L07, FR-H08–FR-L09 FR-H09–FR-L11, EIR-H01–EIR-L03	Presentation Layer	Presentation (Client)	RoutePlannerView
			PoiDetailView
			MapComponent
			AudioPlayer
			ApiClient
FR-H01, FR-L06 FR-H03, FR-H08 FR-L08, FR-L09	Backend API Gateway Layer	API Boundary (Backend)	RouteController
			PoiController
			RequestValidator
			ErrorMapper
			ValidationResult
FR-H01, FR-H02, FR-L02 FR-H03, FR-L03 FR-H04, FR-H06 FR-H07, FR-H08 PR-H01	Backend Services Layer	Core Services	ItineraryService
			PoiService
			RoutingService
			ContentService
			PlanRanker
			HeuristicPlanRanker
			ItineraryBuilder
			MonteCarloItineraryPlanner
			RouteAssembler
		Shared Model	Poi
			PoiContent
			Itinerary
			DayPlan
			RoutePlan
			RouteSegment
			GeoPoint
			MediaAsset
			Language
FR-H02, FR-L02 FR-H06, PR-H01	Integration Layer	Integration	OsrmClient
			OsrmRouteResponse
FR-H02, FR-H04, FR-L04 FR-H05, FR-L05, PR-H01	Data Layer	Data Access	PoiRepository
			MediaRepository
			ContentRepository
			AudioAssetResolver

Table 1: Traceability matrix mapping system requirements to HLD layers, to LLD elements.

1.2 Object design trade-offs

During the design of GUIDE, several architectural and object-level trade-offs were considered to balance usability, performance, security, and system reliability under practical constraints. GUIDE operates as both a user-facing web application and a computationally intensive routing system, which introduces competing quality attributes that cannot be simultaneously maximized.

Design decisions were therefore guided by realistic usage scenarios, expected user interaction patterns, and runtime limitations. In cases where improving one quality attribute would negatively impact another, explicit prioritization was applied to favor overall system effectiveness and user experience. The following trade-offs document these decisions and clarify the rationale behind selecting one design objective over its alternative.

1.2.1 Usability vs. Functionality

One of the primary goals of GUIDE is to minimize user effort in travel planning. Users are expected to provide only high-level constraints (city, duration, maximum distance, category preferences) without interacting with complex configuration panels. For this reason, the system avoids exposing low-level routing parameters or advanced optimization controls that may confuse non-technical users.

Although additional configuration options (e.g. manual route ordering, traffic sensitivity parameters) could increase functionality, they would significantly complicate the user interface and reduce accessibility. Therefore, GUIDE prioritizes simplicity and clarity over maximal configurability, aiming to keep the system intuitive while still providing meaningful itinerary generation.

1.2.2 Efficiency vs. Accuracy

Route generation involves computationally expensive operations, especially when multiple candidate destinations and multi-day grouping constraints are considered. While it would be possible to implement highly sophisticated optimization algorithms that attempt to compute globally optimal solutions, such approaches would increase response time and negatively affect user experience.

GUIDE prioritizes responsiveness, targeting route generation within a few seconds for typical inputs. As a result, heuristic-based filtering, ranking, and day-based allocation strategies are preferred over exhaustive optimization methods. This design choice ensures interactive performance while maintaining sufficient practical accuracy for travel planning purposes.

1.2.3 Security vs. Usability

GUIDE is designed as a session-based system without persistent user accounts. This decision simplifies the user experience by eliminating registration and login requirements, reducing barriers for first-time users. However, certain usability features, such as saved itineraries and personalized history tracking, are sacrificed in favor of privacy and simplicity.

At the same time, secure communication (HTTPS/TLS) and strict input validation are enforced at the API boundary. Rate limiting and request validation mechanisms are included in the API gateway layer to protect backend services from misuse, even though such mechanisms may occasionally restrict repeated rapid requests from legitimate users.

1.2.4 Reliability vs. Compatibility

GUIDE prioritizes system reliability and deterministic runtime behavior. To achieve this, third-party content providers (e.g., description sources, text-to-speech generation services) are excluded from runtime dependencies. All POI metadata and audio assets are prepared offline and served locally during execution.

Although integrating real-time traffic APIs or dynamic POI updates could increase compatibility with external ecosystems, such integrations would introduce additional failure points and unpredictable latency. Therefore, GUIDE emphasizes controlled, self-hosted components (backend services, routing engine, static datasets) to ensure stable and reliable operation.

1.2.5 Automation vs. User Control

GUIDE is designed to automatically generate optimized multi-day itineraries from high-level user constraints. A fully automated approach could maximize planning efficiency by strictly enforcing algorithmic decisions and minimizing user intervention during itinerary construction and routing.

However, fully automated itinerary generation can reduce user trust and may fail to reflect subjective preferences (e.g., personal interest, visiting priorities, or must-see destinations) that cannot be accurately inferred from input parameters alone. For this reason, GUIDE prioritizes user control by allowing interactive modifications after route generation. Users can remove or reorder POIs and trigger iterative replanning until they are satisfied with the result.

This design choice improves transparency and user satisfaction at the cost of potentially reducing global optimality, since user edits may override the most efficient route ordering. Nevertheless, supporting manual adjustment is essential to keep GUIDE flexible and aligned with real-world travel planning behavior.

1.3 Interface documentation guidelines

In this report, all classes, attributes, and methods are documented in a consistent and structured format in order to ensure clarity, traceability, and architectural alignment with the High-Level Design (HLD). The low-level design refines the layered and service-oriented architecture defined in the HLD and translates it into an implementation-ready software components.

The LLD remains independent from specific programming language frameworks, while still being detailed enough to guide direct implementation.

Comment conventions:

During implementation, all classes and public methods defined in this Low-Level Design are expected to be documented using structured documentation comments. Documentation comments will follow widely accepted software documentation standards, the Google Developer Documentation Style Guide practices [5].

These comments will describe the responsibility of each class or method, its parameters, return values, and relevant constraints. This practice ensures consistency between design-level documentation and source code, improves maintainability, and supports traceability from the Low Level Design and other reports to the implementation.

Naming conventions:

- Class names follow the CamelCase format and start with an uppercase letter (e.g., *ItineraryService*, *PoiRepository*, *OsrmClient*).
- Attribute and method names follow camelCase format and start with a lowercase letter (e.g., *generateRoute()*, *maxDistance*, *visitDuration*).
- Constants are written in uppercase with underscores when required (e.g., *MAX_TRIP_DAYS*, *DEFAULT_ROUTE_TIMEOUT*).

Class interface format:

Each class definition in Section 3 is documented using the following structure:

- Class name
- Brief explanation of the class responsibility
- Attributes (with type and visibility)
- Methods (with return type and parameters)
- Additional explanation when necessary

Unless otherwise specified, attributes are assumed to be private and accessed through controlled methods. Method signatures are presented in a language-independent format. Validation, error handling, and integration details are described conceptually rather than tied to a specific framework implementation.

Architectural alignment:

The Low-Level Design refines the layered architecture defined in the High-Level Design by decomposing each architectural layer into more concrete and implementation-oriented component groups. This refinement does not introduce new architectural layers; instead, it provides a detailed breakdown of responsibilities to support implementation and maintainability.

Specifically, the HLD layers are decomposed as follows:

- **Presentation Layer (HLD)** is refined into **Presentation Components (LLD)**, which include client-side modules responsible for user interaction and visualization.
- **Backend API Gateway Layer (HLD)** is refined into the **API Boundary Component (LLD)**, responsible for request validation, routing, and protection mechanisms.
- **Backend Services Layer (HLD)** is refined into **Core Application Services (LLD)** and **Integration Components (LLD)**, separating business logic from routing and external service communication.
- **Data Layer (HLD)** is refined into **Data Access Components (LLD)**, which manage POI metadata, static media assets, and storage abstraction.

This refinement ensures that each low-level class supports a clearly defined architectural responsibility while remaining fully aligned with the layered structure defined in the High Level Design report.

UML and documentation standards:

Class interfaces and subsystem decompositions follow UML guidelines for structural modeling [6]. Documentation practices aim to preserve consistency between requirements, architecture, and implementation. References and citations, where applicable, are formatted in accordance with IEEE citation standards [7].

Example class documentation format:

class RouteService

Description
Coordinates route generation by interacting with the routing engine and organizing selected POIs into a feasible travel plan.
Attributes
private OsrmClient osrmClient private ItineraryBuilder itineraryBuilder private int maxTripDays
Methods
public RouteResponse generateRoute(List<Poi> pois, TravelConstraints constraints)

1.4 Engineering standards

In this report, we follow commonly accepted software engineering documentation practices to keep the low-level design consistent, traceable, and implementation-ready.

- **UML-based modeling:** Class interfaces and component responsibilities are documented using UML-oriented conventions in line with IEEE-recommended software documentation practices. This includes a clear separation of concerns across presentation, API boundary, core service, integration, and data access components, as well as a consistent representation of domain entities (e.g., POI, Itinerary, RoutePlan, TravelPreferences) [6].
- **Consistent LLD structure:** The document follows a structured low-level design organization aligned with IEEE software documentation standards, where architectural decisions are refined through package decomposition, package-level explanations, and detailed class interfaces. This structure supports traceability from requirements to implementation level components [8, 9].
- **Requirement alignment and traceability:** Low-level components are designed to directly support the functional and performance requirements defined in the Project Specification Report. Requirement traceability and verification considerations are aligned with IEEE 830 and ISO/IEC/IEEE 29148 standards for requirements specification and lifecycle consistency [8, 9].
- **Security and privacy awareness:** Design decisions consider secure communication, input validation at the API boundary, and privacy-by-default principles. System-level quality attributes such as security, reliability, and usability are aligned with the ISO/IEC 25010 software product quality model [10].
- **Testability and verification support:** The low-level design explicitly considers testable components, mockable dependencies, and integration boundaries to support systematic verification and validation. These considerations are aligned with IEEE 829-2008 and ISO/IEC/IEEE 29119-3 standards for software test documentation and test design specification [11, 12].
- **Citation and referencing:** External resources and standards referenced in the report are cited using IEEE-style referencing, consistent with academic reporting practices [7].

- **Implementation framework and runtime:** The backend is implemented using **Python 3.12** and the **FastAPI** web framework [13], served via the **Uvicorn** ASGI server [14]. FastAPI provides automatic OpenAPI schema generation, native asynchronous request handling, and built-in request/response validation via Pydantic. The frontend is built with **React 18** [15] and bundled using **Vite** [16], communicating with the backend exclusively through the documented REST API. Database access is performed through the **Supabase** Data API (PostgREST) [17] using the `supabase-py` async client.
- **Containerization and local deployment:** The OSRM routing engine is containerized using **Docker** [18] and managed via a `docker-compose.yml` configuration. The container handles OSM data preprocessing (extract, partition, customize) on first startup and serves the routing API on `localhost:5000`. The backend and frontend are run directly on the host during development. The full local stack requires three concurrent processes: the OSRM Docker container, the Uvicorn backend server, and the Vite development server.

1.5 Definitions, acronyms, and abbreviations

The following terms and abbreviations are used throughout this document:

- **API**: Application Programming Interface.
- **DTO**: Data Transfer Object.
- **HLD**: High-Level Design.
- **LLD**: Low-Level Design.
- **POI**: Point of Interest.
- **OSM**: OpenStreetMap.
- **OSRM**: Open Source Routing Machine.
- **TTS**: Text-to-Speech.
- **UI**: User Interface.
- **UX**: User Experience.
- **HTTPS**: Hypertext Transfer Protocol Secure.

2 Packages

This section presents the package-level decomposition of GUIDE. The package structure directly follows the hybrid *Layered* and *Service-Oriented* architecture defined in the High-Level Design (HLD) report. The goal of this decomposition is to separate responsibilities clearly, support maintainability, and ensure that each package can be implemented and tested with minimal coupling.

At the highest level, GUIDE is organized into two main component groups:

- **Backend**: Contains the API boundary, core services, integration components, and data access packages.
- **Presentation**: Provides user interaction and visualization components.

Figure 1 illustrates the overall package structure and the dependency relationships between these components.

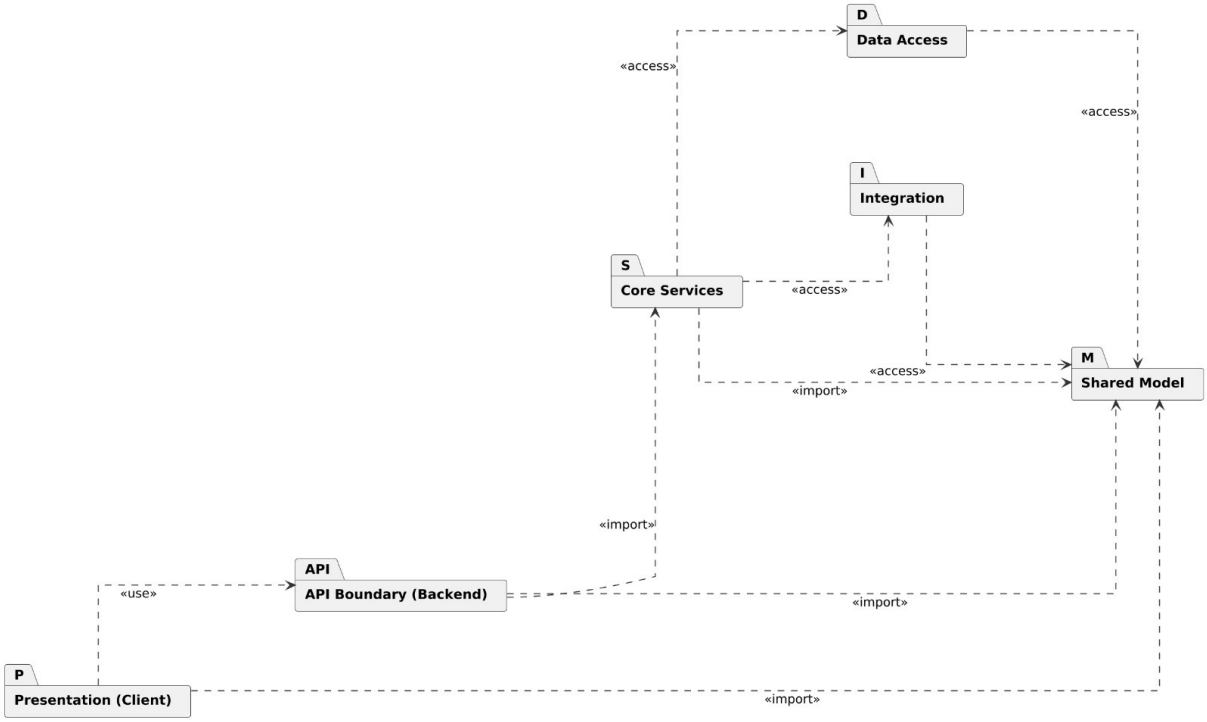

Figure 1: Package overview of GUIDE.

2.1 Backend Packages

The Backend package contains all components responsible for processing user requests, generating itineraries, orchestrating routing, and retrieving curated destination content. The backend is designed to be **stateless** at the service level, meaning that backend services do not retain user-specific session state or previously generated itineraries between requests. Each request contains all information required for processing, including user preferences, constraints, and (for replanning) the current itinerary and user edits. This design supports scalability, predictable behavior under concurrent use, and simplified deployment.

At the Low-Level Design stage, backend functionality is decomposed into concrete packages that directly correspond to the architectural layers defined in the HLD. Each package is described below in detail.

2.1.1 API Boundary

The API Boundary package acts as the controlled entry point to the backend and corresponds to the Backend API Gateway Layer defined in the HLD. It is responsible for validating incoming requests, applying protection mechanisms, and dispatching requests to internal services.

In the Low-Level Design, this package includes controller and validation components such as *RouteController*, *PoiController*, *RequestValidator*, *ValidationResult*, and API-level error and warning representations (e.g., *ApiErrorResponse*, *ApiWarning*).

Typical responsibilities include:

- Input validation and schema checks for route generation and replanning requests
- Request routing to core backend services
- Uniform error and response formatting

2.1.2 Core Services

The Core Services package implements the main business logic of GUIDE and corresponds to the Backend Services Layer in the HLD. Services in this package operate exclusively on validated inputs and coordinate the end-to-end workflow from user preferences to a structured itinerary.

This package includes primary application services such as *ItineraryService*, *PoiService*, *RoutingService*, and *ContentService*. In addition, supporting service components such as *ItineraryBuilder*, *PlanRanker*, and *RouteAssembler* are used to modularize itinerary construction, POI ranking, and route assembly logic.

Core services are designed to remain stateless and communicate through well-defined interfaces, ensuring testability and a clear separation of concerns.

2.1.3 Integration

The Integration package isolates communication with external or self-hosted subsystems from core business logic. In the current scope of GUIDE, this package primarily contains routing-related integration components.

Low-level integration components include *OsrmClient* and *OsrmRouteResponse*, which are responsible for constructing routing queries, executing HTTP requests to the OSRM engine, and normalizing routing responses into internal data structures.

This isolation ensures that changes in routing configuration or response formats can be handled without impacting core service logic.

2.1.4 Data Access

The Data Access package provides structured and read-only access to POI metadata and static media assets. It abstracts underlying storage mechanisms to allow future evolution without affecting the service layer.

In the Low-Level Design, this package includes repository-oriented components such as *PoiRepository*, *ContentRepository*, *MediaRepository*, and *AudioAssetResolver*, which provide controlled access to curated datasets and pre-generated multimedia content.

Media retrieval is designed to tolerate partial availability; if some image or audio assets are missing for a POI, the system returns the available subset and the presentation layer degrades gracefully.

2.1.5 Shared Model

The Shared Model defines the core data structures that represent the fundamental travel planning concepts of GUIDE and are shared across backend services, integration components, and the API boundary.

The shared model includes conceptual entities such as *Poi*, *Itinerary*, *DayPlan*, *RoutePlan*, and *RouteSegment*. These entities encapsulate the core business concepts of the system and are independent of implementation details or specific framework concerns.

In addition to domain entities, GUIDE defines boundary-level Data Transfer Objects (DTOs) such as *RouteRequest*, *RouteResponse*, *ReplanRequest*, and *PoiQuery*. These DTOs are used to exchange structured data across the API boundary and to decouple external request and response formats from internal domain representations.

Together, the domain entities and DTO definitions form a shared model that ensures a consistent representation of core data structures across presentation, API boundary, core service, integration, and data access components.

2.1.6 Presentation

The Presentation package represents the user-facing layer of GUIDE. It is responsible for collecting user inputs, visualizing routes and POIs, and presenting generated itineraries in an interactive and user-friendly manner.

At the Low-Level Design stage, the presentation layer is decomposed into concrete view and component packages such as *RoutePlannerView*, *PoiDetailView*, *MapComponent*, *AudioPlayer*, and *ApiClient*.The presentation layer communicates with the backend exclusively through public API endpoints and does not contain business logic.

2.2 Frontend Packages

While the Presentation package defines the architectural responsibility of the client-side layer, the actual frontend implementation is realized as a React-based single-page application composed of modular JavaScript and CSS artifacts. The Figure 2 diagram presents the concrete frontend file structure and runtime wiring relationships.

The diagram presents the concrete frontend implementation of GUIDE as a modular React-based single-page application. Unlike the architectural Presentation layer defined earlier, this model focuses on implementation-level artifacts, including entry points, page-level views, layout components, reusable UI elements, API communication modules, and utility helpers. Each page and component encapsulates its corresponding CSS module, reflecting a component-scoped styling strategy that improves maintainability and visual isolation.

The wiring relationships highlight how *App.jsx* orchestrates route-based view composition, how page modules interact with the centralized API client for backend communication, and how map rendering responsibilities are delegated to *RouteMap.jsx* and associated Leaflet utilities [19]. This separation clarifies the distinction between architectural responsibilities and concrete frontend implementation structure, ensuring traceability from the Presentation package to the actual codebase.

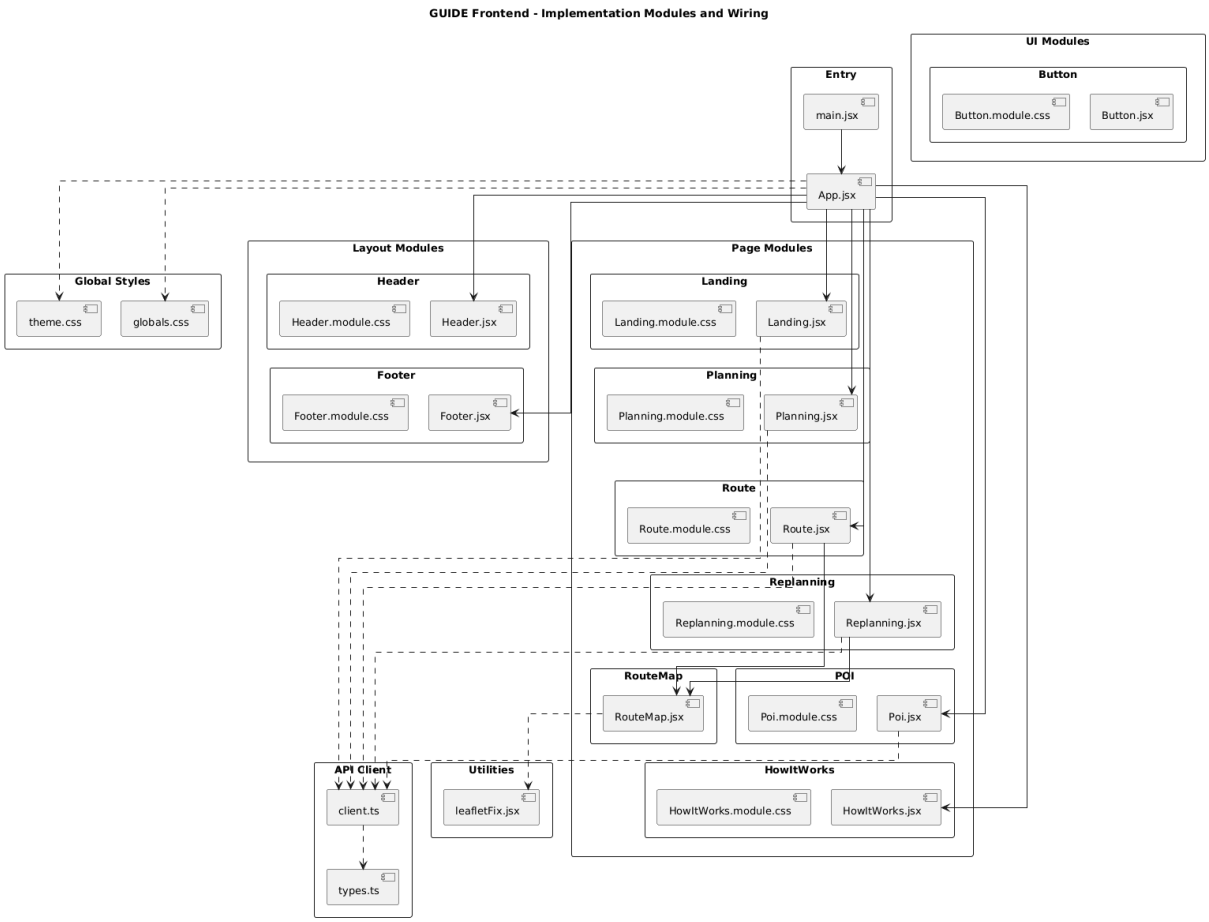

Figure 2: Frontend implementation modules and runtime wiring structure of GUIDE.

2.2.1 Entry

The Entry module contains the application bootstrap files (*main.jsx* and *App.jsx*). These files initialize the React application, configure routing, and define top-level layout composition. All page modules are mounted through the App component.

2.2.2 Global Styles

The Global Styles module includes shared stylesheet definitions (*globals.css* and *theme.css*). These files define application-wide typography, color variables, layout rules, and design tokens. Unlike component-level styles, these files are applied globally.

2.2.3 Layout Modules

Layout modules contain structural UI components such as *Header* and *Footer*. These components provide consistent visual framing across all pages and are reused throughout the application. Each layout component maintains its own scoped CSS module.

2.2.4 Page Modules

Page modules represent route-level views of the application, including *Landing*, *Planning*, *Route*, *Journey*, *RouteMap*, *Replanning*, *Poi*, and *HowItWorks*. Each page encapsulates user interaction logic and composes lower-level UI components.

These modules act as orchestration layers between user input, API calls, and visual presenta-

tion.

2.2.5 UI Modules

UI modules contain reusable visual components such as *Button*. These components are presentation-focused and do not contain business logic. They are styled using dedicated CSS modules and reused across multiple page modules.

2.2.6 API Client

The API Client module (*client.ts* and *types.ts*) encapsulates communication with backend endpoints. It defines request/response structures and abstracts HTTP calls from page components, improving separation of concerns.

2.2.7 Utilities

Utility modules provide helper functions that support third-party integrations or minor cross-cutting concerns. For example, *leafletFix.jsx* adapts Leaflet's default icon configuration for correct map rendering behavior.

Unlike backend components, frontend modules are not documented with detailed class interface specifications in this report. The frontend implementation is based on React functional components and client-side composition patterns, which do not define stable object-oriented interfaces in the same sense as backend services. Since most frontend components primarily handle view rendering, state management, and event-driven interactions rather than encapsulating domain-level business logic, documenting them using full UML-style class interfaces would not provide additional architectural value. Instead, their structure and wiring relationships are presented at the module level to maintain clarity while avoiding unnecessary diagram complexity.

3 Class Interfaces

This section documents the main class interfaces of the GUIDE system. Before presenting detailed class interface definitions, the overall static structure of the system is introduced through UML class diagrams in Figure 3 and Figure 4.

The class diagrams illustrate the organization of classes into packages and their relationships, providing a high-level view of the system structure at the Low-Level Design stage. Separate diagrams are used to present the core backend classes and the shared model in order to improve readability and to avoid diagram overcrowding.

Individual class interfaces are documented in detail in Appendix B. Classes are grouped according to their architectural responsibilities (API Boundary, Core Services, Integration, Data Access, and Frontend Presentation). Each class definition includes a brief responsibility description, key attributes, and method signatures in a language-independent format.

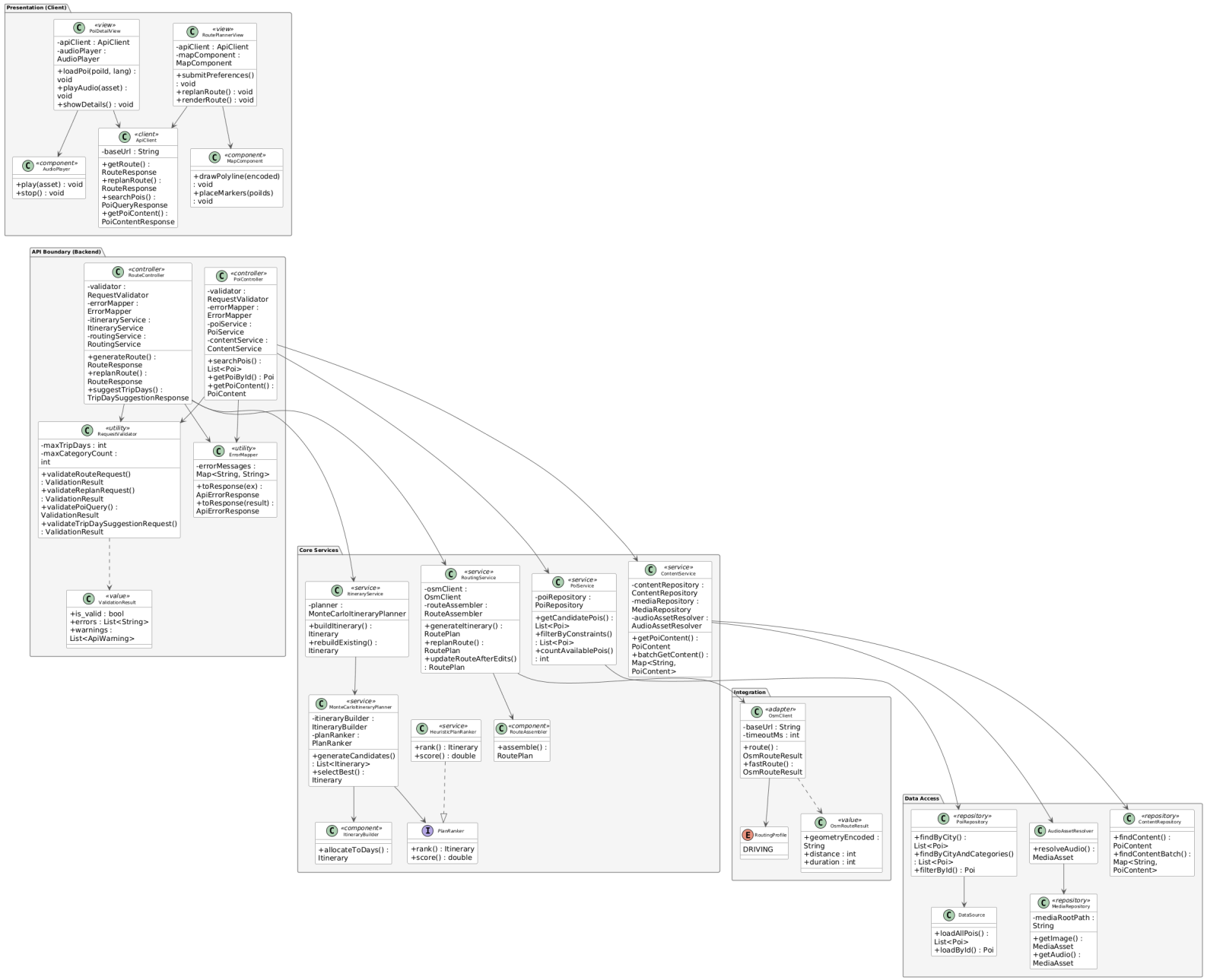

Figure 3: Class Diagram for GUIDE

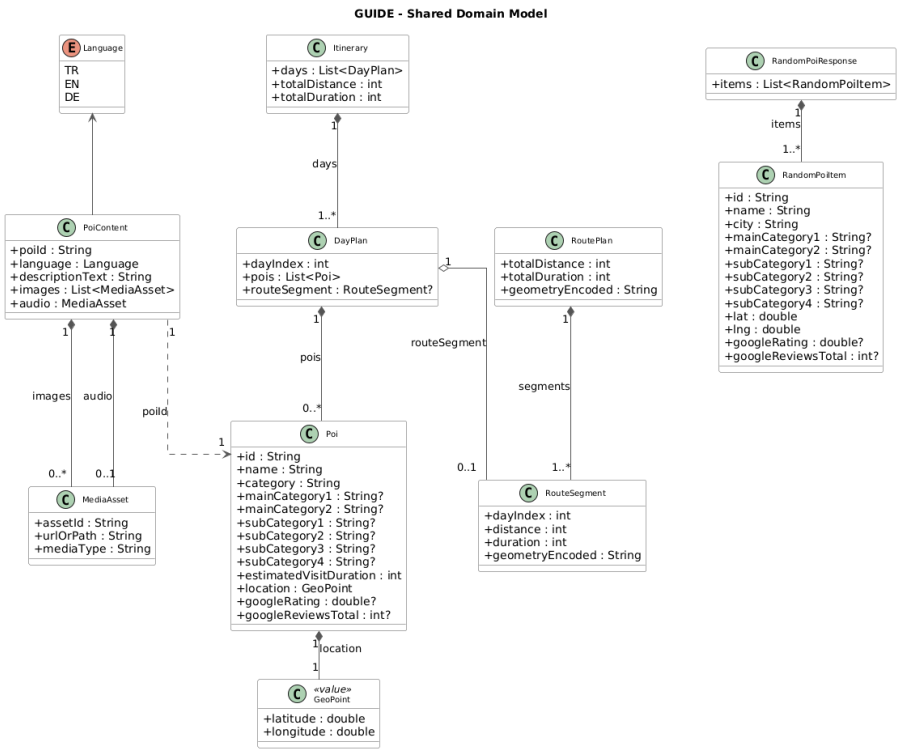

Figure 4: UML class diagram showing the shared model of the GUIDE system.

These diagrams are intended to provide a high-level, static view of the system structure by highlighting class groupings and dependency boundaries. Detailed class responsibilities, attributes, and method signatures corresponding to these diagrams are specified in Appendix B.

4 Behavioral Design

This section describes the runtime behavior of GUIDE by focusing on how user preferences are transformed into a feasible, multi-day itinerary and associated routes. Activity and sequence diagrams are used to illustrate the orchestration of planning, routing, and content delivery components, as well as the iterative replanning process is triggered by user edits. The models highlight how validated inputs propagate through stateless services, how routing requests are delegated to the OSRM integration, and how itinerary and POI content results are assembled and returned to the presentation layer.

4.1 Activity Models

In GUIDE, runtime behavior is captured using two separate activity models in order to clearly distinguish between the planning-oriented workflow and the on-site content consumption workflow. Although both activities are part of the same user journey, they differ significantly in terms of triggering conditions, system responsibilities, and iteration patterns.

By separating these two activities, the design avoids overloading a single model with heterogeneous concerns and provides a clearer representation of GUIDE's planning logic and content delivery behavior.

4.1.1 Complete Route Planning Flow

Figure 5 summarizes the end-to-end workflow of route planning. The user first provides required parameters (destination, trip duration, distance constraints, and category filters), which are validated to ensure feasibility before processing. After submission, the backend validates the request, constructs a feasible itinerary, and requests route geometry from the routing engine. If the user modifies suggested POIs, the system iterates by regenerating and re-rendering an updated route until the user is satisfied.

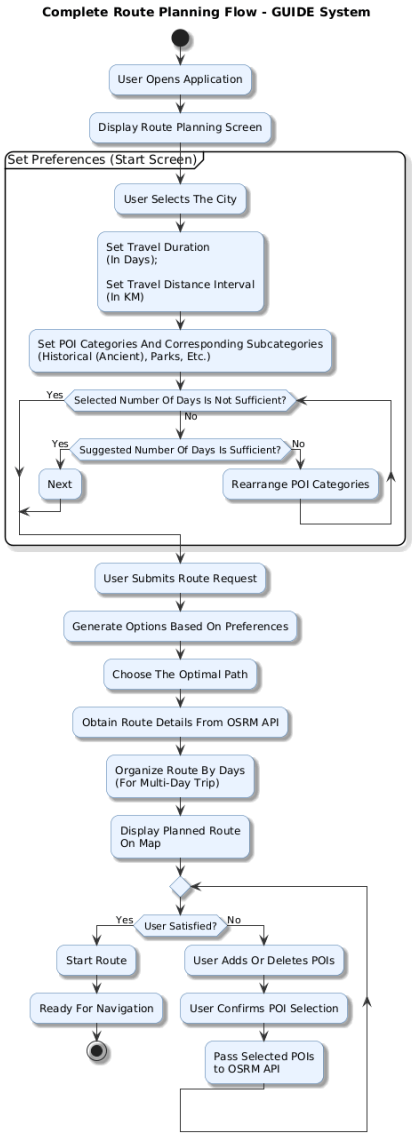

Figure 5: Complete route planning flow in GUIDE.

4.1.2 POI Information Retrieval Flow

Figure 6 describes how GUIDE retrieves POI details and optionally provides voice guidance. Textual content, images, and multilingual audio references are served from the local database at runtime. If audio guidance is available for the selected language, the client plays the corresponding pre-generated audio; otherwise, the system continues in text-only mode.

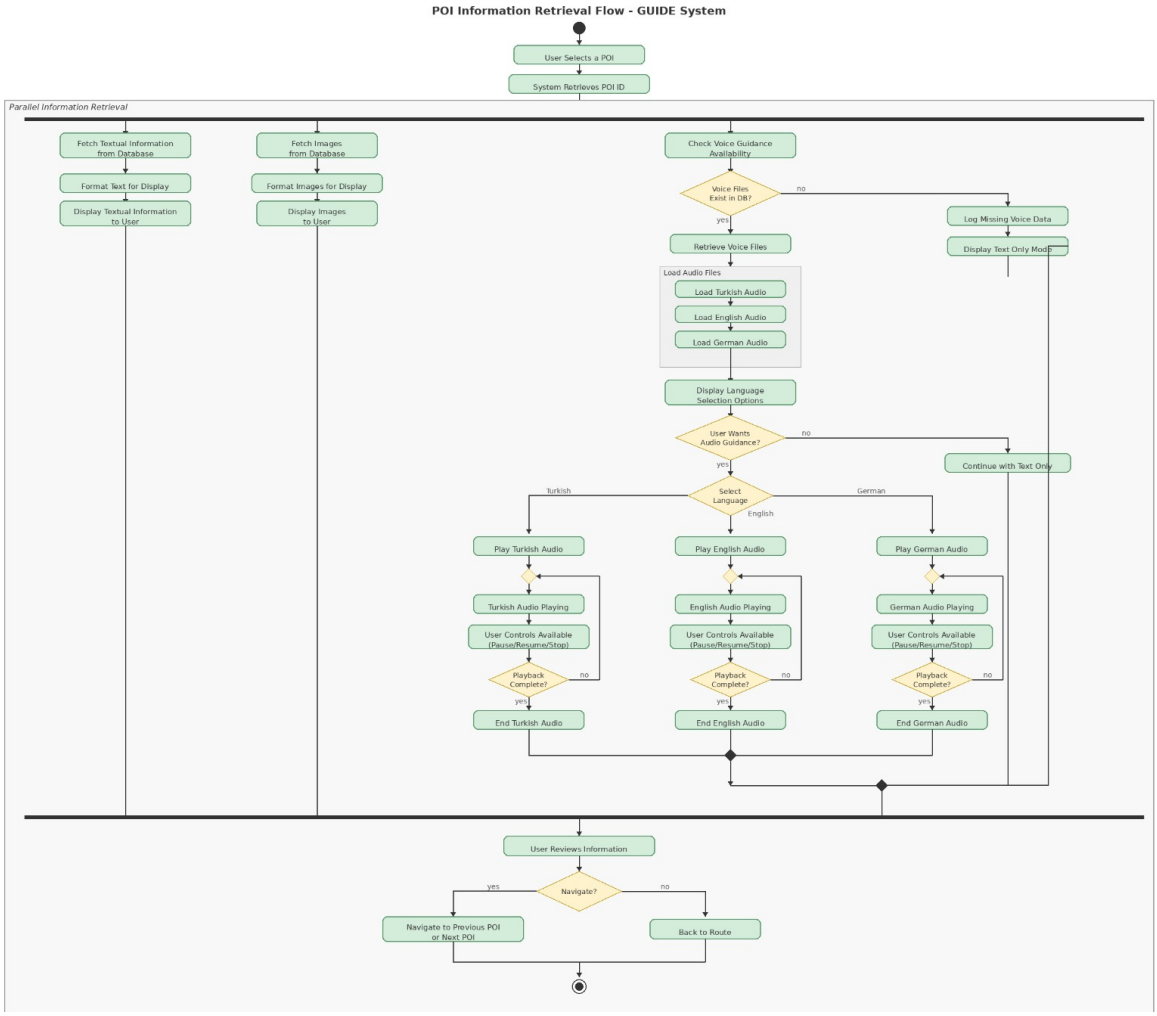

Figure 6: POI information retrieval flow in GUIDE.

4.2 Sequence Models

Sequence diagrams detail the message-level interactions between the client, the API boundary, core services, data access components, and the routing integration. They also show where validation, error mapping, and optional behaviors (e.g., audio playback) occur.

4.2.1 Input Collection Sequence Diagram

Figure 7 shows how user preferences (inputs) are submitted from the client to the backend. The API boundary validates the request and dispatches the workflow to the backend service.

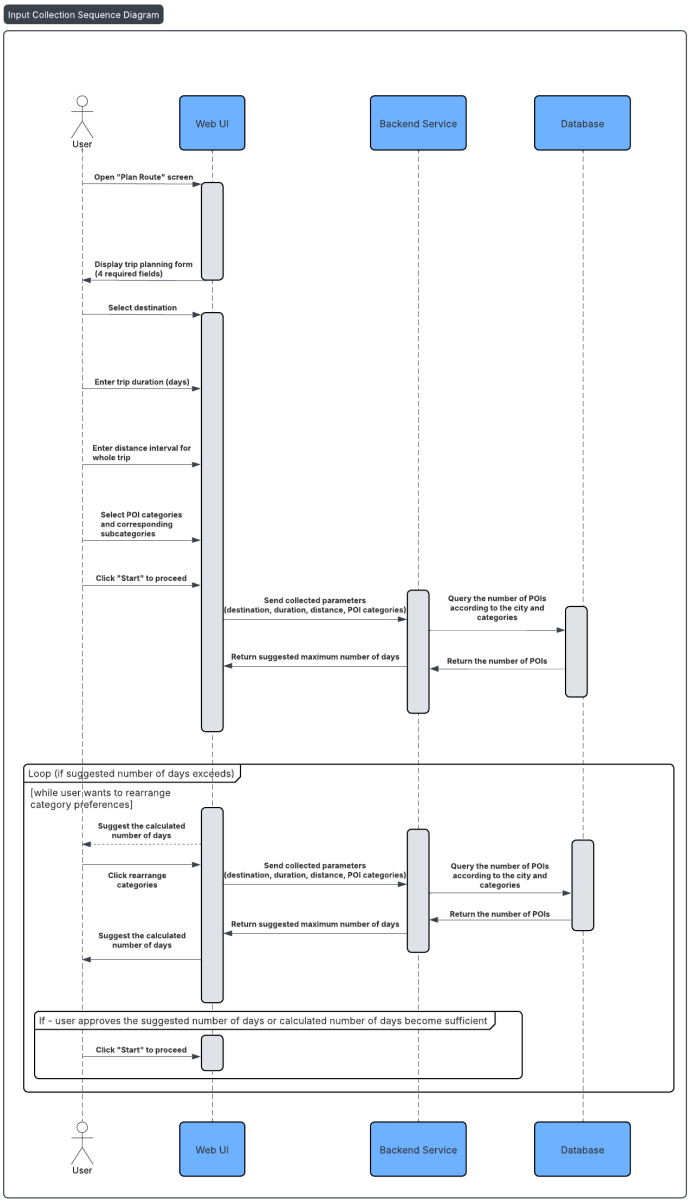

Figure 7: Sequence diagram showing user input submission in GUIDE.

4.2.2 POI Details and Voice Guidance Display

Figure 8 illustrates the interaction flow when a user requests POI details. The client requests descriptive content from the backend. The content service retrieves text and image references from the local dataset and resolves the correct pre-generated audio asset when available. The client then presents the content and triggers optional audio playback based on language selection and asset availability.

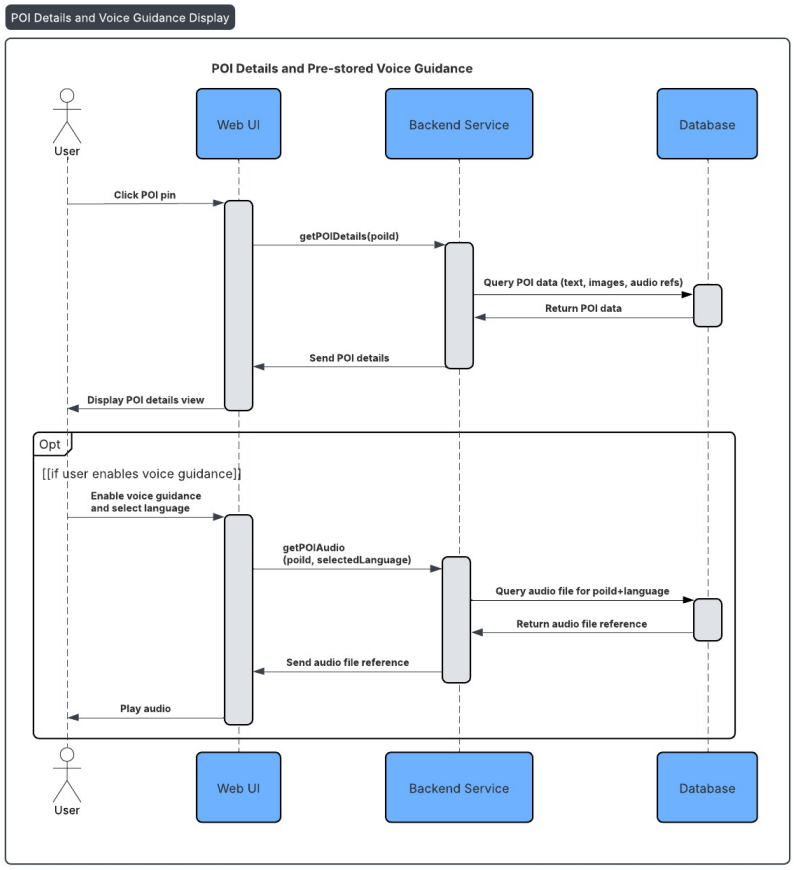

Figure 8: Sequence diagram showing POI detail retrieval and optional audio playback.

4.2.3 Route Generation and Replanning Sequence

Figure 9 illustrates the central runtime interaction pattern of GUIDE, capturing both initial plan generation and the iterative replanning loop triggered by user edits. This sequence represents the primary coordination flow between the presentation layer, backend services, routing integration, and map visualization components.

The interaction begins when the user initiates plan generation through the presentation layer. The backend service processes the request by retrieving a filtered POI pool, generating candidate daily route combinations, ranking these plan candidates using the heuristic scoring function, and delegating route geometry computation to the OSRM integration. The selected daily plan and its corresponding route geometry are then returned to the client for visualization.

The lower part of the diagram highlights the replanning loop, which is a defining characteristic of GUIDE. When the user adds, removes, or reorders POIs within a daily plan, the client submits a replanning request that includes the current itinerary and the user edits. Because the backend is stateless, all required context is explicitly provided in the request. The backend applies edits deterministically, recomputes only the affected route segments via OSRM, re-evaluates the plan structure when necessary, and returns an updated plan without maintaining session state.

This sequence diagram captures the main execution flow of the system, showing how GUIDE supports interactive daily-plan updates while maintaining clear service boundaries and stateless

processing.

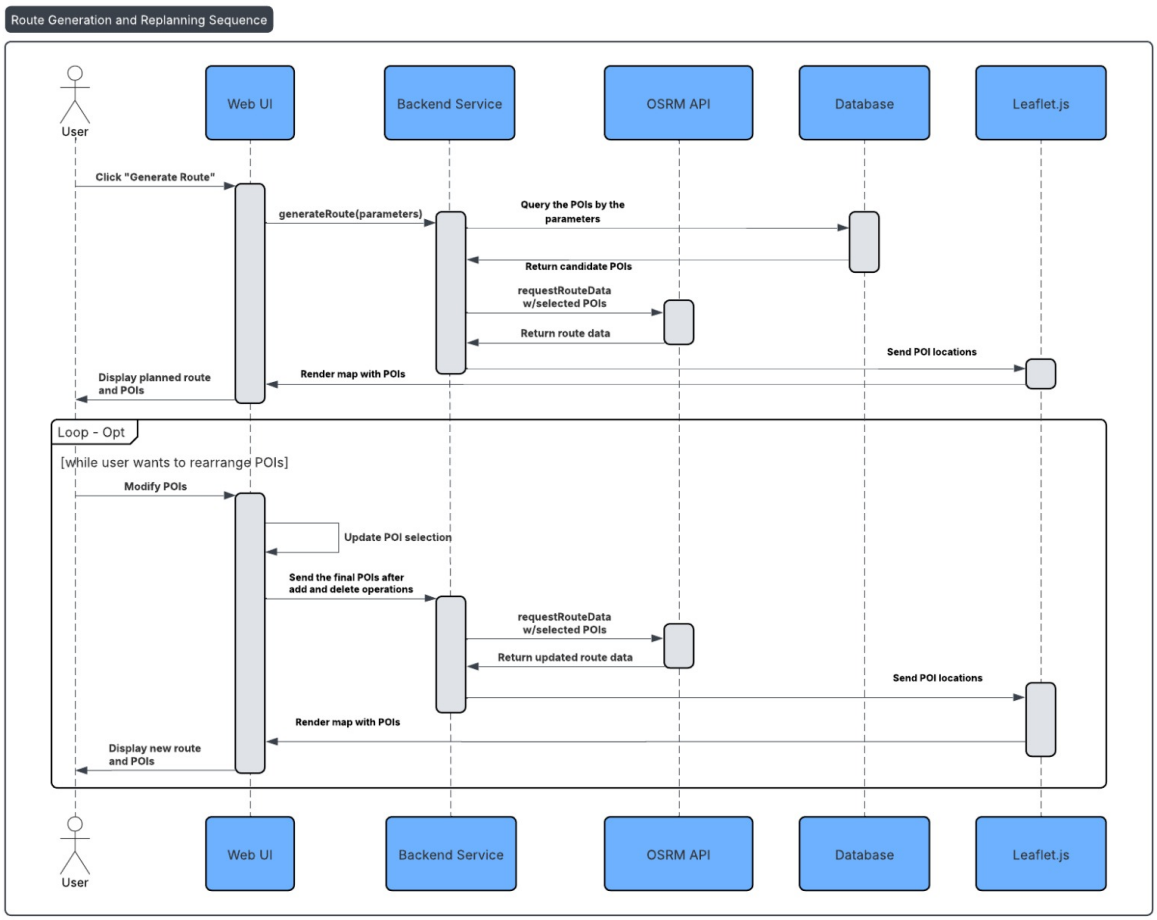

Figure 9: Sequence diagram showing route generation and iterative replanning after user edits.

5 Data Structures

This section describes the core data structures used throughout the GUIDE system. The focus is on the static representation of domain concepts and API-facing data objects that form the shared data model of the system. These structures define stable contracts between architectural layers and provide the foundation for itinerary planning, routing, and content delivery.

The core domain data model captures the fundamental travel planning concepts of GUIDE and is independent of specific implementation or framework details. These domain entities are shared across backend services, integration components, and the API boundary as part of the shared model.

The shared model is designed to be expressive yet stable, enabling deterministic planning and replanning operations while remaining suitable for serialization and transport when required.

5.1 Data Transfer Objects

Data Transfer Objects (DTOs) define the structured payloads exchanged across the API boundary between the presentation layer and backend services. DTOs are used to carry user inputs,

planning results, and replanning context without embedding business logic.

Examples of DTOs include *RouteRequest*, *RouteResponse*, *ReplanRequest*, and *PoiQuery*. These objects encapsulate user preferences, constraints, and itinerary representations required for request processing, while remaining decoupled from internal service implementations.

DTOs are designed to be serializable, explicit, and self-contained, supporting the stateless nature of backend services.

Figure 10 presents the static data model of GUIDE, focusing on the structure and relationships of domain entities and API-facing Data Transfer Objects (DTOs). The diagram emphasizes how itinerary planning, replanning, routing, and content delivery data are represented and exchanged across architectural layers.

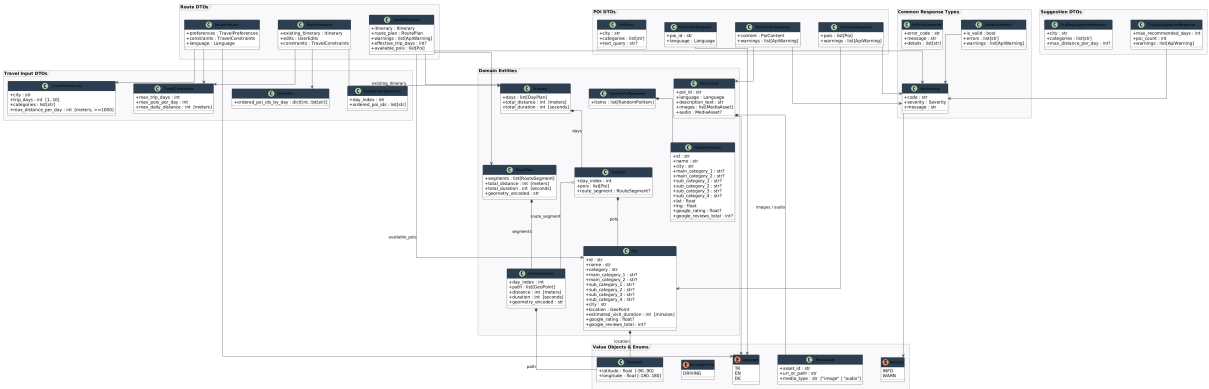

Figure 10: Static data model diagram of GUIDE.

The model distinguishes between core domain entities (e.g., *Poi*, *Itinerary*, *DayPlan*, and *RouteSegment*) and boundary-level DTOs used for request and response payloads (e.g., *RouteRequest*, *RouteResponse*, and *ReplanRequest*). This separation clarifies the role of DTOs as structured carriers of input parameters, planning results, and replanning context, while keeping domain entities focused on business concepts.

Additionally, the model highlights how routing outputs are grouped into day-level segments and how media and multilingual content are associated with POIs for presentation.

Overall, this data model diagram provides a concise reference for understanding the data contracts that bind together planning logic, routing integration, and user interaction in GUIDE.

5.2 Data Types and Size Considerations

In this subsection, the primary data types that are used across GUIDE and practical size considerations for API payloads, in-memory representations, and media assets could be found. The goal is to make data contracts explicit, support performance requirements, and guide implementation choices such as validation limits, serialization formats, and caching strategy.

5.2.1 General conventions

The following conventions, unless the opposite is stated, apply throughout the system:

- **Units:** Distances are represented in **meters** and durations in **seconds** at routing/integration boundaries (consistent with OSRM). UI may display derived units (km/min).

- **Identifiers:** Entity identifiers (e.g., *poiId*) are ASCII strings.
- **Coordinates:** *GeoPoint* uses WGS84 decimal degrees as `double`.
- **Encoding:** Route geometry is transported as a compact **encoded polyline** string (*geometryEncoded*) to reduce payload size.
- **Collections:** Lists are ordered unless explicitly stated otherwise.

5.2.2 Core data structures and field types

Table 2 summarizes the most important domain entities and DTOs, their key fields, and type constraints relevant for validation and implementation.

Table 2: Core data structures, key field types, and constraints.

Structure	**Key Fields (Type)**	**Constraints / Notes**
GeoPoint	*latitude: double*, *longitude: double*	WGS84 degrees. Validate ranges: lat $\in [-90, 90]$, lon $\in [-180, 180]$.
Poi	*id: String*, *name: String*, *category: String*, *location: GeoPoint*, *estimatedVisitDuration: int*	*estimatedVisitDuration* in minutes. *category* must be one of supported categories for the selected city/dataset.
TravelPreferences	*city: String*, *tripDays: int*, *categories: List<String>*, *maxDistancePerDay: int*	*tripDays* $\in [1, MAX_TRIP_DAYS]$. *maxDistancePerDay* in meters (UI may input km, converted).
TravelConstraints	*maxTripDays: int*, *maxPoisPerDay: int*, *maxDailyDistance: int*	Constraint fields act as guardrails. Used by *RequestValidator* for feasibility checks.
Itinerary	*days: List<DayPlan>*, *totalDistance: int*, *totalDuration: int*	Totals are aggregated. *totalDistance* meters, *totalDuration* seconds.
DayPlan	*dayIndex: int*, *pois: List<Poi>*, *routeSegment: RouteSegment*	*dayIndex* starts from 1. *pois* size should be $\leq$ *maxPoisPerDay*.
RoutePlan	*segments: List<RouteSegment>*, *totalDistance: int*, *totalDuration: int*, *geometryEncoded: String*	*geometryEncoded* is polyline (compact). Total geometry may also be derived by concatenating segments.
RouteSegment	*dayIndex: int*, *path: List<GeoPoint>*, *distance: int*, *duration: int*	*path* is optional. In API responses, only *geometryEncoded* is returned, while *path* is omitted (or null). The client may decode *geometryEncoded* into a *List<GeoPoint>* for map rendering. The backend may use decoded points transiently during processing, but they are not persisted.
PoiContent	*poiId: String*, *descriptionText: String*, *images: List<MediaAsset>*, *audio: MediaAsset*	*descriptionText* may be empty if unavailable. *audio* may be null when not provided for language/POI.
MediaAsset	*assetId: String*, *urlOrPath: String*, *mediaType: String*	*mediaType* must be one of {*image*, *audio*} (or an enum in implementation).
RouteRequest	*preferences: TravelPreferences*, *constraints: TravelConstraints*, *language: Language*	Validated at API boundary. *language* must be one of {*TR*, *EN*, *DE*}.

Structure	Key Fields (Type)	Constraints / Notes
ReplanRequest	*existingItinerary: Itinerary*, *edits: UserEdits*, *constraints: TravelConstraints*	Supports a stateless backend. Existing itinerary must be validated for integrity (day indexes, POI ids, counts).
RouteResponse	*itinerary: Itinerary*, *routePlan: RoutePlan*, *warnings: List<ApiWarning>*	Warnings used for non-fatal feasibility issues (e.g., POIs removed).

Scope note: Table 2 focuses on the primary domain entities and main API contracts that directly participate in itinerary construction, routing, and replanning workflows. Supporting DTOs and auxiliary types such as *ApiWarning*, *ApiErrorResponse*, *PoiQuery*, *PoiQueryResponse*, *PoiContentRequest*, *PoiContentResponse*, *UserEdits*, *DayReorderOperation*, and the *Language* enum are included in the data model diagram for completeness, but are not repeated here in order to preserve readability and focus on the core planning data structures.

5.2.3 Recommended validation limits

To preserve interactive response time and prevent oversized requests, the API boundary enforces practical limits. These limits are consistent with GUIDE's expected usage (typical 1–3 day trips in Türkiye) and can be refined based on dataset statistics.

- **MAX_TRIP_DAYS:** 10 (hard upper bound for route planning requests)
- **MAX_POIS_PER_DAY:** 9 (prevents infeasible day schedules)
- **MAX_TOTAL_POIS:** 90 (e.g., 10 days $\times$ 9 POIs)
- **MAX_DESCRIPTION_CHARS:** 2000 per POI content entry (truncation allowed)
- **MAX_IMAGE_ASSETS:** 5 per POI (references only; binary not embedded)
- **MAX_VOICE_DURATION_SECONDS:** 120 per POI audio asset (maximum 2 minutes per voice guidance)

Minimum validity and feasibility requirements:

In addition to upper bounds, the API boundary enforces lower-bound and feasibility constraints to prevent logically valid but practically infeasible requests.

- Numeric counts and durations must be non-negative.
- *tripDays* must be at least 1.
- *maxDistancePerDay* must be ≥ 1000 meters to allow feasible routing.
- At least two POIs must match the selected filters; otherwise the request is rejected or adjusted.

5.2.4 API Payload Size Considerations

API responses are designed to remain compact in order to preserve interactive performance and comply with the 5-second response target for typical 1–3 day trips.

- Route geometry is transmitted as an **encoded polyline** string (*geometryEncoded*) to minimize payload size.
- Distances and durations are represented as integers (meters and seconds), reducing serialization overhead.
- Images and audio files are not embedded in API responses; only references (*urlOrPath*) are transmitted.

A typical 1–3 day itinerary contains 15–25 POIs. In such cases:

- Structured JSON response size is expected to remain below approximately **100–300 KB**.
- Even in worst-case scenarios (maximum POI/day constraints), payload size remains well below common HTTP response limits.

GUIDE does not implement application-level caching. No backend memory cache or distributed cache layer is used. Any caching behavior is limited to default HTTP/browser mechanisms and is not explicitly controlled by the backend system.

5.2.5 Media Size Considerations

GUIDE operates on a carefully curated dataset of approximately 2,500–3,000 POIs, assembled from multiple publicly available sources and enriched through extensive manual filtering, categorization, content refinement, and media preparation efforts carried out by the project team. Media assets (images and audio files) dominate total storage usage, while metadata and textual descriptions remain comparatively small. Media assets are not stored within the primary application database. Instead, images and audio files are hosted on external object storage (e.g., AWS S3 or equivalent cloud storage), and only their references (URLs/paths) are stored in the backend data layer. This design reduces database size, improves scalability, and enables efficient content delivery [20, 21].

Image Assets:
Each POI is limited to:

$$MAX_IMAGE_ASSETS = 5$$

Images are stored as static objects in external cloud storage (e.g., AWS S3), and are not embedded in JSON payloads. Only their URLs are returned via the API. Assuming an average optimized web image size of 300–500 KB:

Worst-case upper bound (theoretical):

$$3{,}000 \times 5 \times 400KB \approx 6GB$$

However, in practice:

- Many POIs include fewer than 5 images,
- Image resolutions are optimized for web delivery,

Therefore, the realistic expected image footprint is approximately **0.8–1.5 GB**.

Audio Assets (Voice Guidance):
Audio guidance is limited to:

$$MAX_VOICE_DURATION_SECONDS = 120$$

Audio is planned to be provided for a large portion of the dataset. Similar to images, audio files are stored in external object storage and are served via URL references. The backend does not store or stream binary audio data directly. At minimum, at least half of POIs are expected to include three audio assets (e.g., one per supported language), and the remaining POIs are targeted to include **at least one audio asset** whenever possible.

Assuming 96 kbps MP3 encoding:

$$96kbps \times 120seconds \approx 1.44MBperaudioasset$$

Let N be the total number of POIs. Under the stated coverage assumption:

$$audiofiles \approx (0.5N \times 3) + (0.5N \times 1) = 2N$$

Thus, total audio storage can be estimated as:

$$2N \times 1.44MB = 2.88NMB$$

For $N \approx 2{,}500$–$3{,}000$ POIs:

$$2.88 \times 2{,}500 \approx 7.2GB \quad to \quad 2.88 \times 3{,}000 \approx 8.6GB$$

Thus, audio storage is conservatively estimated at **7.2–8.6 GB** under maximum-duration assumptions. In practice, the realized footprint may be lower if average audio duration is below 120 seconds and/or if lower bitrates are used.

5.2.6 Textual Content Size Consideration

POI descriptions are stored as plain UTF-8 encoded text. Textual content and POI metadata are stored in a relational database managed through Supabase (PostgreSQL + PostgREST) [17]. This allows efficient querying, filtering, and structured access to POI information while keeping media storage decoupled from the database layer. Each POI entry is limited to 2000 as MAX_DESCRIPTION_CHARS. In practice, average description length is expected to be approximately 800–1200 characters.

Assuming an average of 1,000 characters per POI and a dataset size of 2,500–3,000 POIs:

$$1000 bytes \times 3000 \approx 3MB$$

Under worst-case assumptions (2,000 characters per POI), total textual content size remains below:

$$2000 \times 3000 \approx 6MB$$

Therefore, textual content storage remains negligible compared to audio and image assets and does not pose a scalability risk.

Overall, total media storage (images + audio) is expected to remain within **8–10 GB**, depending on final audio coverage, average voice duration, and image resolution. This estimate remains compatible with the infrastructure constraints of the project and available development environments.

Media assets are stored in external cloud object storage and retrieved on demand via URL references. They are not loaded into application memory except during client-side consumption (e.g., image rendering or audio playback). The backend operates only on lightweight metadata and media references, ensuring efficient memory usage and scalability.

6 Constraints

The design and implementation of GUIDE are influenced by a set of technical, operational, and academic constraints that directly shape architectural decisions, system scope, and implementation strategies. This section summarizes the constraints that are relevant at the Low-Level Design stage and explains how they affect system behavior and structure.

A more detailed discussion of project-wide constraints and their broader impacts is provided in the *Project Constraints and Impacts Document*, available at this link.

6.1 Technical Constraints

Performance:

- Route generation is required to complete within approximately 5 seconds for typical trips of 1–3 days in order to support an interactive user experience. This requirement favors heuristic-based planning approaches and motivates the use of a locally deployed OSRM instance to minimize network latency.
- The system is designed for moderate concurrent usage typical of academic deployment environments and does not target large-scale high-concurrency production traffic.

Geographic Scope:

- Routing and POI datasets are limited to Türkiye. Supporting additional regions would require separate OSM preprocessing pipelines and curated POI datasets, which exceed the scope of the current project.

- The current system design assumes a mid-scale dataset (2,500–3,000 POIs). Significant dataset expansion (e.g., nationwide or multi-country scale) would require re-evaluation of storage, indexing, and ranking strategies.

Map and Road Network Data Constraints:

- Road network coverage and routing feasibility depend on OpenStreetMap (OSM) data quality. Missing, outdated, or incorrectly tagged roads and restrictions may lead to sub-optimal or infeasible routes, even when the routing engine is functioning correctly.

Real-Time Data Limitations:

- GUIDE does not integrate real-time traffic, road condition, or dynamic routing data. All route computations are based on static OpenStreetMap data.
- Incorporating real-time traffic APIs would introduce additional cost, external dependencies, and runtime uncertainty that are incompatible with the project scope.

POI Operational Information:

- The system does not track dynamic POI operational status such as opening hours, temporary closures, or event-based accessibility changes.
- Maintaining continuously updated operational data for a large POI set is not feasible within the constraints of an academic project.

POI Labeling Model Constraints:

- POI labeling is performed through a hybrid approach combining manual curation and semi-automated enrichment processes. Categories and semantic groupings are primarily assigned based on curated datasets and domain-specific filtering rules rather than fully automated embedding-based similarity models.
- In the dataset preparation pipeline, external sources and lightweight text processing techniques (including optional LLM-assisted refinement) are used to standardize descriptions and improve category consistency. This approach prioritizes controllability and interpretability over fully automated semantic embedding pipelines.
- Labeling quality is therefore bounded by dataset curation quality, category definition clarity, and consistency of applied filtering rules. Factors such as source heterogeneity, category ambiguity, and manual preprocessing decisions may affect labeling accuracy.

Optimization and Heuristic Constraints:

- Route and POI scheduling decisions are produced using heuristic search methods (e.g., Monte Carlo-style sampling or comparable heuristic approaches) to preserve interactive response time [22].
- Because the planning problem is combinatorial, the system does not guarantee globally optimal itineraries. Output quality is therefore constrained by heuristic parameters and improving solution quality may increase runtime.

6.2 Infrastructure and Deployment Constraints

Routing Infrastructure:

- Local OSRM deployment requires approximately 2–4 GB of memory for processing Türkiye-scale OSM datasets. This requirement is satisfied by the development environments available to the project team.

Content Storage:

- GUIDE relies on externally stored POI metadata and media assets rather than keeping all content within the local application environment. Structured POI metadata and textual content are managed through Supabase, while image and audio assets are hosted in cloud object storage and accessed via stored references [17, 20].
- For an estimated dataset size of 2,500–3,000 POIs, total media storage requirements are expected to range between 8–10 GB, dominated by audio and visual assets. However, this footprint corresponds primarily to external cloud storage usage rather than local application storage, since the backend serves metadata and media references instead of storing all assets on the host system.

Map Visualization:

- The frontend uses Leaflet for interactive map visualization [19]. Base map tiles are retrieved from external OpenStreetMap tile servers in accordance with usage policies [1]. Consequently, GUIDE is subject to external tile server API usage limits, rate restrictions, and availability constraints defined by the OSM tile service.

Memory Usage:

- Media assets (images and audio) are not stored within the backend runtime environment. Instead, they are hosted in external cloud object storage and accessed on demand via URL references. As a result, media files are not loaded into backend memory and are only consumed at the client side during rendering or playback.
- The backend operates primarily on lightweight metadata structures retrieved from Supabase, along with request-scoped planning data used during itinerary generation and ranking. These structures remain small in size and are released after each request due to the stateless design of the system.
- A minimum development environment assumption of 8 GB RAM is considered sufficient for local OSRM deployment and backend execution. Memory usage is dominated by routing computations and in-memory planning operations rather than media handling.

6.3 Functional Scope Constraints

Language Support:

- Multilingual content and audio guidance are limited to Turkish, English, and German. The graphical user interface is provided in English only.

Audio Guidance Coverage (TTS Cost Constraints):

- Pre-generated text-to-speech (TTS) audio is provided only for a curated subset of POIs due to cost and generation effort. When audio assets are not available for a POI or language, GUIDE falls back to text-only presentation.

POI Content Sources:

- POI descriptions and media are derived from a broad set of publicly available sources, including international heritage databases, municipal and governmental portals, cultural institution websites, open encyclopedic platforms, and royalty-free media repositories. The dataset was constructed using a combination of manual curation and semi-automated scraping workflows.

  Representative data sources include OpenStreetMap (OSM) for geographic structure [23], Google Places API for POI metadata enrichment (e.g., ratings, popularity, and images) [24], and curated textual content refined through semi-automated processing pipelines. External services such as Wikipedia may be used as auxiliary references during offline dataset preparation [25], but are not directly queried at runtime.

  Only publicly accessible and license-compatible content was incorporated and processed into a static dataset to avoid runtime dependency and licensing conflicts.

- Source heterogeneity (in formatting, completeness, and update frequency) can introduce inconsistencies, missing fields, or content quality variation. Although filtering and normalization steps are applied, the final dataset quality remains bounded by the reliability and accessibility of the original sources.

Data Currency:

- POI content is curated during development and is not updated automatically at runtime. Automated content refresh pipelines are outside the scope of this project.

User Interaction Model:

- GUIDE operates without persistent user accounts. All interactions are session-based, which simplifies the architecture but prevents long-term personalization and itinerary persistence.

- Users may manually modify or override generated itineraries within a session, ensuring user control over automated planning results.

Dataset Integrity:

- POI metadata and media assets are curated manually and validated during dataset preparation. Runtime validation does not perform automated content verification against external sources.

6.4 Academic and Development Constraints

Project Timeline:

- Development is constrained by the academic calendar, spanning September 2025 to April 2026.

Team Capacity:

- The project is developed by a four-member undergraduate team with an estimated capacity of approximately 60 person-hours per week.

Technology Choices:

- Open-source tools and libraries are preferred wherever feasible. Proprietary services (e.g., Google Text-to-Speech [26] and Google Places API [24]) are used selectively when high-quality or scalable alternatives are not sufficiently available in the open-source ecosystem.

Scalability Assumption:

- GUIDE is designed for moderate-scale academic deployment. Large-scale production deployment, high-concurrency scenarios, or significant dataset expansion would require architectural re-evaluation beyond the current project scope.

7 Error Conditions and Edge Cases

This section summarizes non-happy-path scenarios considered in the design of GUIDE. Unlike systems that rely on free-text input or multiple external real-time APIs, GUIDE is intentionally designed to minimize runtime errors through controlled user inputs, static datasets, and clear service boundaries.

Most potential error conditions are therefore prevented at the interface level or handled through graceful degradation rather than system termination. Detailed test cases derived from these scenarios will be documented in the Test Plan.

7.1 Input and Validation Errors

User input is collected through structured UI components (e.g., dropdowns, sliders, predefined category lists), which significantly reduces the risk of syntactic input errors. Free-text input is avoided by design.

Nevertheless, the API boundary performs defensive validation to detect:

- Missing required fields,
- Trip durations outside globally defined limits,
- Daily distance values outside predefined system-level bounds.

Syntactic and global constraint violations are rejected before planning logic is executed. However, feasibility conditions that depend on city-specific POI distribution or filter combinations can only be determined during the planning phase.

In such cases, the planner returns partial results or explicit warnings instead of treating the situation as a hard validation error.

7.2 Planning and Feasibility Edge Cases

Edge cases in GUIDE primarily arise from feasibility constraints rather than input format errors.

Typical examples include:

- Very restrictive filter combinations yielding too few POIs,
- Extremely low distance limits preventing feasible routing,
- Replanning scenarios where too many POIs are locked into a single day,
- Sparse geographic distributions producing less balanced routes.

These situations are treated as domain-level constraints, not system failures. The planner may return partial results, reduce candidate sets, or include explicit warnings. The system is designed to always return a structurally valid itinerary whenever possible.

7.3 Integration and Routing Failures

Routing relies on a locally deployed OSRM instance. Potential failures include:

- Routing engine unavailability,
- Timeout during route computation,
- Invalid or incomplete routing responses.

In such cases, route generation is terminated in a controlled manner and a consistent error response is returned. Incomplete or inconsistent route data is never exposed to the user.

7.4 Data Availability and Content Gaps

Because the dataset is compiled from heterogeneous public sources, some POIs may lack:

- Descriptive text,
- Image assets,
- Multilingual audio guidance.

These cases are treated as non-fatal content gaps.

GUIDE applies a graceful degradation strategy: missing audio falls back to text-only presentation, missing images result in reduced-media display, and incomplete descriptive fields do not block itinerary generation.

Content incompleteness does not affect system stability or planning correctness.

7.5 Robustness Strategy

GUIDE is intentionally designed to reduce error-prone scenarios through:

- Controlled user input mechanisms,
- Static dataset usage,
- Local routing deployment,
- Stateless request processing.

As a result, most edge cases relate to feasibility or content completeness, not unexpected runtime failures. Even under constrained conditions, the system remains operational and prioritizes graceful degradation over hard failure.

8 Testability Considerations

This section explains how GUIDE supports systematic testing at both component and system levels. While the architectural design enables extensive unit and integration testing, it is important to acknowledge that itinerary quality evaluation contains inherently subjective elements.

GUIDE generates structured travel routes rather than exact results. Therefore, correctness can be verified deterministically at the structural and constraint level, whereas perceived route quality depends on user preference and domain interpretation.

8.1 Unit-Testable Components

Core planning logic is decomposed into deterministic, side-effect-free components operating at the daily plan level. Classes such as *ItineraryBuilder*, *PlanRanker* implementations, and *RouteAssembler* operate on explicit inputs and produce structured outputs.

The ranking process is applied to complete daily route candidates rather than individual POIs. Given a fixed set of candidate routes and deterministic scoring parameters, the ranking behavior remains reproducible and unit-testable.

These components can be unit-tested to verify:

- Correct allocation and grouping of POIs within daily routes,
- Compliance with predefined distance constraints,
- Deterministic daily-plan ranking behavior for fixed scoring inputs,
- Correct transformation of routing outputs into internal route structures.

Although overall plan quality may vary depending on embedding-based labeling performance, the deterministic scoring and transformation logic itself remains testable and verifiable.

8.2 Ranking Evaluation Challenges

Daily plan ranking in the system is based on a multi-parameter heuristic scoring function rather than a single objective criterion. The ranking metric combines several weighted factors such as total route distance, target-distance proximity, category diversity, and structural balance.

The final scoring formulation and parameter weights were determined empirically through iterative experimentation and comparative observation during implementation.

Since multiple parameters jointly influence the final score, small changes in weight configuration may alter the resulting order of candidate plans. Therefore, ranking behavior reflects design trade-offs rather than an objectively optimal solution.

Embedding models are not used for ranking but solely for POI labeling. As a result, route ordering is independent of embedding similarity scores, and ranking consistency depends only on deterministic scoring parameters.

While structural correctness of the ranking pipeline is fully testable, overall itinerary quality remains dependent on heuristic design choices and parameter calibration rather than mathematically provable optimality.

8.3 Mockable Dependencies

External dependencies are abstracted behind interfaces, including:

- *OsrmClient*,
- *PoiRepository*,
- *ContentRepository*,
- *MediaRepository*.

These components can be replaced with mocks or stubs when necessary, particularly for controlled unit testing scenarios.

However, given the academic project scope and manageable infrastructure requirements, integration tests are primarily executed against real components (e.g., local OSRM deployment and actual dataset snapshots) to validate end-to-end system behavior.

8.4 Integration Testing Scope

Integration testing validates the full request flow from the API boundary through core services and back to the response layer. This includes:

- Initial route generation,
- Replanning after user edits,
- Error-handling paths,
- Graceful degradation under incomplete content.

Integration tests may use a fixed dataset snapshot to ensure reproducibility. This approach enables verification of routing integration, constraint enforcement, and response consistency.

8.5 Test Coverage Expectations

Unit tests are expected to cover core deterministic logic components, with a target coverage range of approximately 65–75% for planning and transformation layers, which is typical for systems combining algorithmic logic and external integrations.

Achieving very high line-coverage percentages is neither fully meaningful nor practically representative for all components of GUIDE. Certain parts of the system, particularly heuristic planning logic and embedding-based ranking, produce valid but non-unique outputs. In such cases, correctness cannot be verified through strict output equality, and artificially increasing coverage may not improve actual system reliability.

For this reason, coverage targets are set at a realistic level, prioritizing deterministic core logic and constraint-validation layers, while evaluating heuristic and model-dependent components through behavioral and scenario-based tests rather than exhaustive line coverage.

9 Glossary

Term	Description
API	Application Programming Interface. A set of endpoints and protocols that enable communication between the frontend and backend components of the GUIDE system.
DTO	Data Transfer Object. A structured object used to transfer data between system layers or across the API boundary without embedding business logic.
HLD	High-Level Design. A design document that describes the overall architecture, subsystem decomposition, and major responsibilities of the GUIDE system.
LLD	Low-Level Design. A detailed design document that refines the HLD by specifying packages, classes, interfaces, and their interactions at the implementation level.
POI	Point of Interest. A location of interest for travelers, such as landmarks, museums, restaurants, or cultural sites, used as core entities in itinerary generation.
OSM	OpenStreetMap. An open-source geographic data platform used as the basis for map visualization and routing data in GUIDE.
OSRM	Open Source Routing Machine. A high-performance routing engine used to compute routes, distances, and travel durations between POIs.
TTS	Text-to-Speech. Technology used to convert textual destination descriptions into spoken audio content for accessibility and guided exploration.
UI	User Interface. The client-side components through which users interact with GUIDE, including forms, maps, and itinerary views.
UX	User Experience. The overall quality of interaction and usability perceived by users when planning and exploring itineraries with GUIDE.
Route Segment	A portion of a route, typically corresponding to a single day of travel, containing path geometry, distance, and duration information.
Itinerary	A structured, multi-day travel plan consisting of ordered POIs, daily groupings, and associated route information.
Shared Model	A set of core entities (e.g., POI, Itinerary, DayPlan) that represent the conceptual data structures and relationships of the GUIDE system independent of implementation details.
API Boundary	The architectural layer is responsible for request validation, error handling, and controlled access between the client and backend services.
Traceability Matrix	A mapping that links system requirements and HLD subsystems to their corresponding LLD packages and classes, supporting requirement coverage and design verification.

Table 3: Glossary of terms used in the GUIDE Low-Level Design report.

A REST API Contract

This appendix documents the complete REST API contract of the GUIDE backend. All endpoints are served under the base path `/api/v1` and communicate via JSON over HTTP. The API is implemented using FastAPI and follows standard HTTP semantics for methods, status codes, and error responses [13].

A.1 Endpoint Overview

Table 4 lists all available endpoints, their HTTP methods, URL paths, and the corresponding controller method that handles each request.

Method	URL	Controller	Description
POST	`/api/v1/routes/generate`	RouteController	Generate multi-day itinerary and route
POST	`/api/v1/routes/replan`	RouteController	Replan itinerary after user edits
POST	`/api/v1/routes/suggest-days`	RouteController	Suggest max feasible trip days
POST	`/api/v1/pois/search`	PoiController	Search POIs by city and categories
POST	`/api/v1/pois/content`	PoiController	Get POI description, images, audio
GET	`/api/v1/pois/{poiId}`	PoiController	Get single POI by ID
GET	`/api/v1/pois/random`	PoiController	Get random POI sample (dataset explorer)

Table 4: GUIDE REST API endpoint overview.

A.2 HTTP Status Codes

Table 5 defines the HTTP status codes returned by the API and the conditions under which each is issued.

Status	Name	Condition
200	OK	Request processed successfully. Response body contains the result.
400	Bad Request	Feasibility constraint violation (e.g., fewer than 2 POIs match the selected filters).
404	Not Found	Requested resource does not exist (e.g., unknown `poiId`).
422	Unprocessable Entity	Request schema or domain constraint violation (e.g., `tripDays` exceeds `MAX_TRIP_DAYS`, missing required field). Returned with a structured error body.
503	Service Unavailable	OSRM routing engine is unreachable or timed out. Route computation could not be completed.

Table 5: HTTP status codes used by the GUIDE API.

Endpoint-specific behavior: Route generation and replanning endpoints (`/routes/generate`, `/routes/replan`) primarily return 422 for constraint violations and 503 when the routing engine is unavailable. POI-related endpoints (e.g., `/pois/{poiId}`) may return 404 when the requested resource does not exist.

All error responses follow the `ApiErrorResponse` structure:

```
{
```

```
  "message": "Not enough POIs match the given filters.",
  "error_code": "INSUFFICIENT_POIS",
  "details": []
}
```

A.3 Sample Request and Response Payloads

A.3.1 POST /api/v1/routes/generate

Request body (RouteRequest):

```
{
  "preferences": {
    "city": "Istanbul",
    "trip_days": 2,
    "categories": ["Fortifications", "Ancient & Archaeology"],
    "max_distance_per_day": 50000
  },
  "constraints": {
    "max_trip_days": 2,
    "max_pois_per_day": 5,
    "max_daily_distance": 50000
  },
  "language": "EN"
}
```

Response body (RouteResponse) — abbreviated:

```
{
  "itinerary": {
    "days": [
      {
        "day_index": 1,
        "pois": [
          {
            "id": "42",
            "name": "Rumeli Fortress",
            "category": "Cultural Heritage",
            "location": { "latitude": 41.085, "longitude": 29.056 },
            "estimated_visit_duration": 90
          }
        ],
        "route_segment": {
          "day_index": 1,
          "distance": 12400,
```

```
          "duration": 1820,
          "geometry_encoded": "_p~iF~ps|U..."
        }
      }
    ],
    "total_distance": 12400,
    "total_duration": 1820
  },
  "route_plan": {
    "segments": [ ... ],
    "total_distance": 12400,
    "total_duration": 1820,
    "geometry_encoded": "_p~iF~ps|U..."
  },
  "warnings": [],
  "effective_trip_days": 1,
  "available_pois": [ ... ]
}
```


A.3.2 POST /api/v1/routes/replan

Request body (ReplanRequest):


```
{
  "existing_itinerary": { ... },
  "edits": {
  "ordered_poi_ids_by_day": {
    "1": ["17", "55", "89"]
  }
}
  "constraints": {
    "max_trip_days": 2,
    "max_pois_per_day": 5,
    "max_daily_distance": 50000
  }
}
```


A.3.3 POST /api/v1/pois/search

Request body (PoiQuery):


```
{
  "city": "Ankara",
  "categories": ["Museum"]
}
```

Response body (PoiQueryResponse) — abbreviated:

```
{
  "pois": [
    {
      "id": "101",
      "name": "Museum of Anatolian Civilizations",
      "category": "Museums",
      "location": { "latitude": 39.940, "longitude": 32.860 },
      "estimated_visit_duration": 120
    }
  ],
  "warnings": []
}
```

A.3.4 POST /api/v1/pois/content

Request body (PoiContentRequest):

```
{
  "poi_id": "101",
  "language": "EN"
}
```

Response body (PoiContentResponse):

```
{
  "content": {
    "poi_id": "101",
    "description_text": "The Museum of Anatolian Civilizations...",
    "images": [
      {
        "asset_id": "101-img-01",
        "url_or_path": "https://storage.example.com/images/101/01.jpg",
        "media_type": "image"
      }
    ],
    "audio": {
      "asset_id": "101-audio-en",
      "url_or_path": "https://storage.example.com/audio/101/en.mp3",
      "media_type": "audio"
    }
  },
  "warnings": []
}
```

B Detailed Class Interfaces

This appendix provides the detailed class interface definitions of GUIDE. Classes are documented using a consistent format (responsibility, key attributes, and method signatures) to support implementation.

B.1 Backend

B.1.1 API Boundary

class RouteController

Description

Handles route generation and replanning endpoints by validating requests, orchestrating itinerary and routing services, and returning a normalized API response (including warnings) in a stateless workflow. Exposed under `/api/v1/routes/*`.

Attributes

private RequestValidator validator

private ItineraryService itineraryService

private RoutingService routingService

private ErrorMapper errorMapper

Methods

public RouteResponse generateRoute(RouteRequest request)

public RouteResponse replanRoute(ReplanRequest request)

public TripDaySuggestionResponse suggestTripDays(TripDaySuggestionRequest request)

class PoiController

Description

Handles POI search, metadata retrieval, and rich content delivery (descriptions, media). Exposed under `/api/v1/pois/*`.

Attributes

private PoiService poiService

private ContentService contentService

private RequestValidator validator

private ErrorMapper errorMapper

Methods

public `List<Poi>` searchPois(PoiQuery query)

public Poi getPoiById(String poiId)

public PoiContent getPoiContent(String poiId, Language lang)

class RequestValidator

Description

Validates incoming API requests through schema checks and domain constraints, and verifies replanning payload consistency for stateless operation.

Attributes

private int maxTripDays

private int maxCategoryCount

private int maxPoisPerDay

private int maxTotalPois

private int minDailyDistanceMeters

Methods

public ValidationResult validateRouteRequest(RouteRequest request)

public ValidationResult validatePoiQuery(PoiQuery query)

public ValidationResult validateReplanRequest(ReplanRequest request)

public ValidationResult validateTripDaySuggestionRequest(TripDaySuggestionRequest request)

class ErrorMapper

Description

Maps internal exceptions and validation outcomes into consistent API error responses.

Attributes

private Map<String, String> errorMessages

Methods

public ApiErrorResponse toResponse(Exception ex)

public ApiErrorResponse toResponse(ValidationResult result)

public `List<ApiWarning>` normalizeWarnings(`List<ApiWarning>` warnings)

class ValidationResult

Description

Represents the outcome of request validation, including fatal errors and non-fatal warnings.

Attributes

private boolean isValid

private `List<String>` errors

private `List<ApiWarning>` warnings

B.1.2 Core Services

class PoiService

Description

Selects candidate Points of Interest (POIs) from the curated dataset based on user preferences (city, categories, exclusion rules) and basic feasibility filters. Produces a candidate POI pool for itinerary planning.

Attributes

private PoiRepository poiRepository

Methods

public `List<Poi>` getCandidatePois(TravelPreferences prefs)

public `List<Poi>` filterByConstraints(`List<Poi>` pois, TravelConstraints constraints)

public int countAvailablePois(String city, `List<String>` categories)

interface PlanRanker

Description

Defines a ranking/scoring strategy for itinerary candidates (day allocations).

Methods

public double score(Itinerary candidate, TravelPreferences prefs, TravelConstraints constraints)

class HeuristicPlanRanker

Description

Baseline scoring implementation for itinerary candidates based on heuristic criteria (e.g., category balance, distance feasibility, visit duration balance).

Methods

public double score(Itinerary candidate, TravelPreferences prefs, TravelConstraints constraints)

class ItineraryService

Description

Constructs a feasible day-by-day itinerary by allocating candidate POIs into daily groups under constraints, and produces a deterministic updated itinerary during replanning based on user edits.

Attributes

private ItineraryBuilder itineraryBuilder

private PlanRanker planRanker

Methods

public Itinerary buildItinerary(`List<Poi>` pois, TravelConstraints constraints)

public Itinerary replan(Itinerary existing, UserEdits edits, TravelConstraints constraints)

class RoutingService

Description

Coordinates route computation via OSRM and assembles results into day-segmented RoutePlan outputs for map visualization; during replanning, recomputes route segments affected by user edits while keeping the backend stateless.

Attributes

private OsrmClient osrmClient

private RouteAssembler routeAssembler

Methods

public RoutePlan generateRoute(Itinerary itinerary, TravelConstraints constraints)

public RoutePlan updateRouteAfterEdits(Itinerary itinerary, UserEdits edits)

class MonteCarloItineraryPlanner

Description

Generates multiple candidate itineraries using a Monte Carlo-style sampling approach and selects the best candidate using a ranking strategy (*PlanRanker*). Uses *ItineraryBuilder* to allocate POIs into day plans under constraints, then scores candidates deterministically based on heuristic criteria.

Attributes

private ItineraryBuilder itineraryBuilder

private PlanRanker planRanker

private int maxIterations

private int randomSeed (optional; for reproducibility)

Methods

public `List<Itinerary>` generateCandidates(`List<Poi>` pois, TravelConstraints constraints)

public Itinerary selectBest(`List<Poi>` pois, TravelConstraints constraints)

public Itinerary selectBestFromCandidates(`List<Itinerary>` candidates, TravelConstraints constraints)

class ContentService

Description

Aggregates POI descriptions, images, and pre-generated multilingual audio references; when assets are missing, returns the available subset to support graceful degradation in the client.

Attributes

private ContentRepository contentRepository

private MediaRepository mediaRepository

private AudioAssetResolver audioAssetResolver

Methods

public PoiContent getPoiContent(String poiId, Language lang)

public `Map<String, PoiContent>` batchGetContent(`List<String>` poiIds, Language lang)

class ItineraryBuilder

Description

Builds a multi-day itinerary by allocating POIs into days and producing an initial schedule.

Methods

public Itinerary allocateToDays(`List<Poi>` pois, TravelConstraints constraints)

class RouteAssembler

Description

Converts routing engine outputs into internal RoutePlan/RouteSegment structures.

Methods

public RoutePlan assemble(Itinerary itinerary, `List<OsrmRouteResponse>` osrmOutputs)

B.1.3 Integration

class OsrmRouteResponse

Description

Represents the normalized subset of OSRM response fields needed by GUIDE.

Attributes

private int distance

private int duration

private String geometryEncoded

class OsrmClient

Description

Handles communication with the OSRM engine by constructing route/trip requests, executing HTTP calls with timeouts, and normalizing OSRM responses into OsrmRouteResponse objects consumed by core services.

Attributes

private String baseUrl

private int timeoutMs

Methods

public OsrmRouteResponse route(`List<GeoPoint>` waypoints, RoutingProfile profile)

public OsrmRouteResponse trip(`List<GeoPoint>` waypoints, RoutingProfile profile)

enum RoutingProfile

Description

Specifies routing mode used by the OSRM engine. Determines which road network and constraints are applied.

Values

DRIVING

B.1.4 Data Access

class PoiRepository

Description

Provides access to curated POI metadata. Supports queries by city, category, tags, and exclusion rules. Abstracts the underlying storage mechanism.

Attributes

private DataSource dataSource

Methods

public `List<Poi>` findByCity(String city)

public `List<Poi>` findByCityAndCategories(String city, `List<String>` categories)

public Poi findById(String poiId)

class MediaRepository

Description

Provides access to static media assets (images and pre-generated TTS audio). Resolves asset paths/URLs based on POI identifiers and language selection.

Attributes

private String mediaRootPath

Methods

public MediaAsset getImage(String poiId)

public MediaAsset getAudio(String poiId, Language lang)

interface DataSource

Description

Abstracts the underlying POI storage mechanism (e.g., JSON files, CSV dataset, database). Provides a generic interface for retrieving structured POI records.

Methods

public `List<Poi>` loadAllPois()

public Poi loadById(String poiId)

class ContentRepository

Description

Provides destination descriptions and image/audio metadata for POIs.

Methods

public PoiContent findContent(String poiId, Language lang)

public `Map<String, PoiContent>` findContentBatch(`List<String>` poiIds, Language lang)

class AudioAssetResolver

Description

Resolves the correct pre-generated audio asset for a POI and language selection.

Methods

public MediaAsset resolveAudio(String poiId, Language lang)

B.1.5 Data Transfer Objects and Core Data Structures (Shared Model Package)

Client-side state note: GUIDE is designed as a session-based system without persistent user accounts. To keep backend services stateless and horizontally scalable, replanning operations are handled using client-side state. This means the client includes the current itinerary (and user edits) in the replanning request. The backend validates the received itinerary and applies edits deterministically to generate an updated plan.

class TravelPreferences

Description

Represents user-selected preferences for itinerary generation.

Attributes

private String city

private int tripDays

private `List<String>` categories

private int maxDistancePerDay

class TravelConstraints

Description

Defines constraints applied during planning and replanning.

Attributes

private int maxTripDays

private int maxPoisPerDay

private int maxDailyDistance

class PoiQueryResponse

Description

Response payload containing POIs matching the query parameters. Used by the client to display search results or candidate POIs.

Attributes

private `List<Poi>` pois

private `List<ApiWarning>` warnings

class PoiContentRequest

Description

Request payload used by the client to retrieve detailed content for a specific POI.

Attributes

private String poiId

private Language language

class PoiContentResponse

Description

Response payload containing descriptive content, images, and an optional audio reference for a POI.

Attributes

private PoiContent content

private `List<ApiWarning>` warnings

enum Severity

Description

Defines severity levels for API warnings and informational messages.

Values

INFO, WARN

class GeoPoint

Description

Represents a geographical coordinate.

Attributes

private double latitude

private double longitude

class Poi

Description

Represents a Point of Interest (POI) in the dataset.

Attributes

private String id
private String name
private String category
private String mainCategory1 (optional)
private String mainCategory2 (optional)
private String subCategory1 (optional)
private String subCategory2 (optional)
private String subCategory3 (optional)
private String subCategory4 (optional)
private GeoPoint location
private int estimatedVisitDuration
private double googleRating (optional)
private int googleReviewsTotal (optional)

class Itinerary

Description

Represents a multi-day structured travel plan.

Attributes

private `List<DayPlan>` days
private int totalDistance
private int totalDuration

class DayPlan

Description

Represents a single day's schedule and its associated route segment.

Attributes

private int dayIndex
private `List<Poi>` pois
private RouteSegment routeSegment

class UserEdits

Description

Represents user modifications applied to an existing itinerary.

Attributes

private Map<Integer, List<String» orderedPoiIdsByDay

class ReplanRequest

Description

User edits are expressed as an ordered mapping of POI identifiers per day, allowing the client to fully control POI ordering during replanning.

Attributes

private Itinerary existingItinerary

private UserEdits edits

private TravelConstraints constraints

B.1.6 Presentation

class RoutePlannerView

Description

Collects user inputs (city, duration, constraints, categories) and triggers itinerary generation by calling backend endpoints. Renders route and itinerary outputs.

Attributes

private ApiClient apiClient

private MapComponent mapComponent

Methods

public void submitPreferences(TravelPreferences prefs)

public void renderRoute(RoutePlan routePlan)

class PoiDetailView

Description

Displays POI details including description, images, and audio playback controls. Requests POI content from the backend when needed.

Attributes

private ApiClient apiClient

private AudioPlayer audioPlayer

Methods

public void loadPoi(String poiId, Language lang)

public void playAudio(MediaAsset audio)

class ApiClient

Description

Client-side HTTP wrapper that communicates with the backend API to request POI search, route generation, replanning, and POI content.

Attributes

private String baseUrl

Methods

public RouteResponse getRoute(RouteRequest req)

public RouteResponse replanRoute(ReplanRequest req)

public PoiQueryResponse searchPois(PoiQuery req)

public PoiContentResponse getPoiContent(PoiContentRequest req)

class MapComponent

Description

Client-side map visualization component responsible for rendering the route geometry and displaying POI markers on the map.

Methods

public void drawPolyline(encoded : String)

public void placeMarkers(pois : List<Poi>)

class AudioPlayer

Description

Client-side audio playback component used to play multilingual POI audio guidance assets and control playback state.

Methods

public void play(asset : MediaAsset)

public void stop()

B.1.7 Additional DTOs, Enums, and Supporting Types

class RouteRequest

Description

Request payload for initial itinerary and route generation.

Attributes

private TravelPreferences preferences

private TravelConstraints constraints

private Language language

class RouteResponse

Description

Response payload containing the generated itinerary and its route plan.

Attributes

private Itinerary itinerary

private RoutePlan routePlan

private `List<ApiWarning>` warnings

private int effectiveTripDays (optional)

private `List<Poi>` availablePois

class TripDaySuggestionRequest

Description

Request payload used after category selection to recommend the maximum feasible trip day count for the selected city and categories (optionally considering daily distance bounds).

Attributes

private String city

private `List<String>` categories

private int maxDistancePerDay (optional)

class TripDaySuggestionResponse

Description

Response payload returning a recommended upper bound for trip days based on POI coverage and constraints.

Attributes

private int maxRecommendedDays

private int poiCount

private `List<ApiWarning>` warnings

class RoutePlan

Description

Structured route output used for map visualization and day-by-day navigation.

Attributes

private `List<RouteSegment>` segments

private int totalDistance

private int totalDuration

private String geometryEncoded

class RouteSegment

Description

Represents a portion of the route (typically one day).

Attributes

private int dayIndex

private `List<GeoPoint>` path (optional; not serialized in API by default)

private int distance

private int duration

class PoiQuery

Description

Represents query parameters for searching POIs.

Attributes

private String city

private `List<String>` categories

private String textQuery

class RandomPoiItem

Description

Lightweight POI representation returned by the random POI endpoint. Uses flat coordinate fields instead of a nested GeoPoint for simpler client-side map rendering.

Attributes

private String id

private String name

private String city

private String mainCategory1 (optional)

private String mainCategory2 (optional)

private String subCategory1 (optional)

private String subCategory2 (optional)

private String subCategory3 (optional)

private String subCategory4 (optional)

private double lat

private double lng

private double googleRating (optional)

private int googleReviewsTotal (optional)

class RandomPoiResponse

Description

Response wrapper returned by the `GET /api/v1/pois/random` endpoint, containing a sampled list of POIs for dataset exploration.

Attributes

private `List<RandomPoiItem>` items

enum Language

Description

Supported language options for content and audio selection.

Values

TR, EN, DE

class MediaAsset

Description

Represents a media resource reference (image/audio).

Attributes

private String assetId

private String urlOrPath

private String mediaType

class PoiContent

Description

Content package returned for a POI, including text, images, and audio reference.

Attributes

private String poiId

private Language language

private String descriptionText

private `List<MediaAsset>` images

private MediaAsset audio

class ApiErrorResponse

Description

Standard error payload returned by the API boundary.

Attributes

private String errorCode

private String message

private `List<String>` details

class ApiWarning

Description

Represents non-fatal issues (e.g., POIs removed due to infeasible constraints or limited dataset coverage).

Attributes

private String code

private Severity severity

private String message

References

[1] OpenStreetMap Foundation. (2023) OpenStreetMap. [Online]. Available: https://www.openstreetmap.org

[2] OSRM Project. (2023) Open Source Routing Machine HTTP API Documentation. [Online]. Available: http://project-osrm.org

[3] QUAD-CORE Team, "Guide – high-level design report," TOBB University of Economics and Technology, Department of Computer Engineering, Tech. Rep. Version 1.0, Jan. 2026, bIL 495 / YAP 495 Graduation Project.

[4] ——, "Guide – project specifications report," TOBB University of Economics and Technology, Department of Computer Engineering, Tech. Rep. Version 1.0, Jan. 2026, bIL 495 / YAP 495 Graduation Project.

[5] Google, "Google developer documentation style guide," https://google.github.io/styleguide/pyguide.html#38-comments-and-docstrings, 2024, accessed: Feb. 2026.

[6] *OMG Unified Modeling Language (UML) Specification, Version 2.5.1*, Object Management Group (OMG) Std., 2017, accessed: Feb. 2026. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[7] *IEEE Editorial Style Manual*, IEEE, 2023, accessed: Feb. 2026. [Online]. Available: https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual

[8] *IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std., 1998.

[9] *IEEE/ISO/IEC 29148 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*, ISO/IEC/IEEE Std., 2018.

[10] *ISO/IEC 25010 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*, ISO/IEC Std., 2011.

[11] *IEEE Std 829-2008 – IEEE Standard for Software and System Test Documentation*, IEEE Std., 2008.

[12] *ISO/IEC/IEEE 29119-3 – Software and Systems Engineering – Software Testing – Part 3: Test Documentation*, ISO/IEC/IEEE Std., 2013.

[13] S. Ramírez, "Fastapi documentation," https://fastapi.tiangolo.com/, 2024.

[14] E. OSS, "Uvicorn asgi server," https://www.uvicorn.org/, 2024.

[15] Meta, "React documentation," https://react.dev/, 2024.

[16] E. You, "Vite documentation," https://vitejs.dev/, 2024.

[17] S. Inc., "Supabase documentation," https://supabase.com/docs, 2024.

[18] D. Inc., "Docker documentation," https://docs.docker.com/, 2024.

[19] Leaflet.js. (2023) Interactive Maps Library for JavaScript. [Online]. Available: https://leafletjs.com

[20] Amazon Web Services, "Amazon s3 - object storage service," https://docs.aws.amazon.com/s3/, 2024, accessed: 2026.

[21] Microsoft Azure Documentation, "Cloud storage patterns for scalable applications," https://learn.microsoft.com/en-us/azure/architecture/patterns/, 2023, accessed: 2026.

[22] R. Rubinstein and D. Kroese, *Simulation and the Monte Carlo Method*. Wiley, 2016.

[23] OpenStreetMap Foundation. Licence/Attribution Guidelines. [Online]. Available: https://osmfoundation.org/wiki/Licence/Attribution_Guidelines

[24] Google, "Google places api," 2023. [Online]. Available: https://developers.google.com/maps/documentation/places/web-service

[25] Wikimedia Foundation. (2023) MediaWiki REST API Documentation. [Online]. Available: https://www.mediawiki.org/wiki/API:REST_API

[26] Google, "Google cloud text-to-speech api," 2023. [Online]. Available: https://cloud.google.com/text-to-speech